

**DETEKSI TINGKAT KANTUK PADA SINYAL EEG DAN EOG
MENGUNAKAN MODEL MAMBA: STUDI PERBANDINGAN
REGRESI DAN KLASIFIKASI**

TUGAS AKHIR

Oleh:

Bintang Fikri Fauzan

122140008



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2026**

ABSTRAK

Deteksi Tingkat Kantuk pada Sinyal EEG dan EOG menggunakan Model

Mamba: Studi Perbandingan Regresi dan Klasifikasi

Bintang Fikri Fauzan

Kantuk saat berkendara merupakan faktor risiko utama kecelakaan lalu lintas yang sulit dinilai secara mandiri oleh pengemudi. Penelitian ini merancang dan mengevaluasi arsitektur model Mamba berbasis *Selective State Space Model* (SSM) untuk mendeteksi tingkat kantuk dari sinyal *Electroencephalogram* (EEG) dan *Electrooculogram* (EOG) menggunakan dataset DROZY (36 rekaman, 14 subjek). Model dievaluasi menggunakan *subject-wise 5-fold cross-validation* dengan segmentasi *sliding window* 30 detik dan normalisasi *Z-Score* per-subjek. Studi perbandingan dilakukan antara dua skenario pelatihan: klasifikasi biner langsung menggunakan *Weighted Cross-Entropy Loss*, dan regresi nilai *Karolinska Sleepiness Scale* (KSS) kontinu (1-9) yang dikonversi ke keputusan biner pada tahap inferensi. Hasil menunjukkan bahwa arsitektur Mamba yang dikembangkan sangat ringan secara komputasi, dengan hanya 62.930 parameter dan beban operasi 0,01 GFLOPS. Pada evaluasi performa, pendekatan klasifikasi (akurasi rata-rata $66,00\% \pm 8,26\%$; *F1-Score* $0,5844 \pm 0,1319$) secara konsisten mengungguli pendekatan regresi (akurasi $57,51\% \pm 6,48\%$; *F1-Score* $0,4676 \pm 0,1566$) dalam mendeteksi kondisi *Drowsy*, meskipun dengan risiko *false alarm* yang lebih tinggi. Tingginya standar deviasi antar *fold* mengindikasikan bahwa performa masih sangat dipengaruhi oleh variabilitas sinyal antarindividu, yang menjadi tantangan utama yang perlu diatasi pada penelitian selanjutnya.

Kata Kunci: Deteksi Kantuk, *Electroencephalogram*, *Electrooculogram*, Model Mamba, Klasifikasi.

ABSTRACT

Detection of Sleepiness Level in EEG and EOG Signals using Mamba Model:

A Comparative Study of Regression and Classification

Bintang Fikri Fauzan

Drowsiness while driving is a primary risk factor for traffic accidents that is difficult for drivers to self-assess. This study designs and evaluates a Mamba model architecture based on the Selective State Space Model (SSM) to detect drowsiness levels from EEG and EOG signals using the DROZY dataset (36 recordings, 14 subjects). The model was evaluated using subject-wise 5-fold cross-validation with 30-second sliding window segmentation and per-subject Z-score normalization. A comparative study was conducted between two training scenarios: direct binary classification using Weighted Cross-Entropy Loss, and continuous KSS value regression (1–9) converted into binary decisions at the inference stage. The results indicate that the developed Mamba architecture is highly computationally efficient, featuring only 62,930 parameters and an operational load of 0.01 GFLOPS. In performance evaluation, the classification approach (average accuracy $66.00\% \pm 8.26\%$; F1-Score 0.5844 ± 0.1319) consistently outperformed the regression approach (accuracy $57.51\% \pm 6.48\%$; F1-Score 0.4676 ± 0.1566) in detecting drowsy conditions, albeit with a higher risk of false alarms. The high standard deviation across folds suggests that performance remains significantly influenced by inter-individual signal variability, posing a primary challenge to be addressed in future research.

Keywords: Drowsiness Detection, Electroencephalogram, Electrooculogram, Mamba Model, Classification

DAFTAR ISI

ABSTRAK	ii
ABSTRACT	iii
DAFTAR ISI	iv
DAFTAR TABEL	viii
DAFTAR GAMBAR	ix
DAFTAR RUMUS	x
DAFTAR KODE	xi
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	5
1.4 Batasan Masalah	5
1.5 Manfaat Penelitian	6
1.6 Sistematika Penulisan	6
BAB II TINJAUAN PUSTAKA	8
2.1 Tinjauan Pustaka	8
2.2 Dasar Teori	12
2.2.1 Klasifikasi Kantuk	12
2.2.2 Sinyal Fisiologis	13
2.2.2.1 Electroencephalogram (EEG)	13
2.2.2.2 Electrooculogram (EOG)	14
2.2.3 Pra-pemrosesan Sinyal	15
2.2.3.1 Segmentasi Sinyal	15

2.2.3.2	Normalisasi Z-Score	16
2.2.4	Augmentasi Data Sinyal	16
2.2.4.1	<i>Gaussian Noise</i>	17
2.2.4.2	<i>Amplitude Scaling</i>	17
2.2.5	<i>Cross Validation</i>	18
2.2.6	Deep Learning	21
2.2.7	State Space Models (SSM)	22
2.2.8	Selective State Space Models	23
2.2.9	Mamba	24
2.2.10	<i>Weighted Cross Entropy Loss</i>	26
2.2.11	<i>Confusion Matrix</i>	27
2.2.12	Evaluasi Kompleksitas Komputasi	29
2.2.12.1	Jumlah Parameter	30
2.2.12.2	GFLOPs	30
2.2.13	<i>Mean Absolute Error (MAE)</i>	31
BAB III	METODE PENELITIAN	32
3.1	Alur Penelitian	32
3.2	Penjabaran Langkah Penelitian	32
3.2.1	Identifikasi Masalah	33
3.2.2	Studi Literatur	33
3.2.3	Penggunaan Dataset	34
3.2.4	<i>Subject Wise K-Fold Cross Validation</i>	35
3.2.5	<i>Preprocessing Data</i>	36
3.2.6	Augmentasi Data	37
3.2.7	Pemodelan dengan Mamba	38
3.2.8	Evaluasi	38
3.3	Lingkungan Pengembangan Tugas Akhir	39
3.3.1	Perangkat Keras (<i>Hardware</i>)	39

3.3.2 Perangkat Lunak (<i>Software</i>)	40
3.4 Metode Pengembangan	41
3.5 Ilustrasi Perhitungan	42
3.5.1 <i>Precision</i>	43
3.5.2 <i>Recall</i>	44
3.5.3 <i>F1-Score</i>	44
3.5.4 <i>Accuracy</i>	44
3.6 Rancangan Pengujian	45
BAB IV HASIL DAN PEMBAHASAN	46
4.1 Hasil Penelitian	46
4.1.1 Akuisisi Dataset	46
4.1.2 Pembagian Dataset	47
4.1.3 Prapemrosesan Data	48
4.1.4 Augmentasi Data	51
4.1.5 Hasil Pelatihan Model	53
4.2 Hasil Evaluasi Kinerja Model	57
4.2.1 Kinerja Pendekatan Regresi	57
4.2.2 Kinerja Pendekatan Klasifikasi	59
4.2.3 Perbandingan Metrik Per-Kelas	61
4.3 Analisis Efisiensi Komputasi	63
4.4 Perbandingan dengan <i>Baseline</i>	64
4.5 Analisis Hasil Penelitian	66
4.5.1 Analisis Karakteristik Regresi dan Klasifikasi	66
4.5.2 Analisis Efektivitas Mekanisme <i>Selective State Space</i>	67
4.5.3 Analisis Variansi dan Pengaruh Karakteristik Subjek	68
BAB V KESIMPULAN DAN SARAN	69
5.1 Kesimpulan	69
5.2 Saran	70

DAFTAR PUSTAKA	71
-----------------------------	-----------

DAFTAR TABEL

Tabel 2.1	Literasi Penelitian Terdahulu	10
Tabel 4.1	Skema Pembagian Subjek pada 5-Fold Cross-Validation . . .	48
Tabel 4.2	Konfigurasi <i>Hyperparameter</i> Pelatihan Model Mamba . . .	53
Tabel 4.3	Hasil Evaluasi 5-Fold Cross-Validation pada Target Regresi	57
Tabel 4.4	Hasil Evaluasi 5-Fold Cross-Validation pada Target Klasifikasi	59
Tabel 4.5	Perbandingan Metrik Per-Kelas Pendekatan Regresi dan Klasifikasi	62
Tabel 4.6	Metrik Efisiensi Komputasi Arsitektur Mamba	63
Tabel 4.7	Perbandingan Kinerja dan Efisiensi Komputasi Model	64

DAFTAR GAMBAR

Gambar 2.1	Ilustrasi <i>K-Fold Cross Validation</i> [28]	19
Gambar 2.2	Pembagian Dataset dengan teknik <i>Subject-wise cross-validation</i>	20
Gambar 2.3	Diagram lanskap <i>Artificial Intelligence</i>	21
Gambar 2.4	Arsitektur Selective State Space Model [6]	23
Gambar 2.5	Diagram Blok Arsitektur Mamba [33]	25
Gambar 2.6	Confusion Matrix	28
Gambar 3.1	Alur Penelitian	32
Gambar 3.2	Pemodelan dengan Mamba	41
Gambar 3.3	Ilustrasi Confusion Matrix	43
Gambar 4.1	Contoh Sinyal dari Kedua Kelas Tingkat Kantuk	47
Gambar 4.2	Perbandingan Sinyal Sebelum & Sesudah Normalisasi ..	50
Gambar 4.3	Sinyal Augmentasi	51
Gambar 4.4	Grafik Loss Pendekatan Regresi	54
Gambar 4.5	Grafik Loss Pendekatan Klasifikasi	55
Gambar 4.6	Confusion Matrix Agregasi Pendekatan Regresi	58
Gambar 4.7	Confusion Matrix Agregasi Pendekatan Klasifikasi	60

DAFTAR RUMUS

Rumus 2.1	Rumus Z-Score	16
Rumus 2.2	Rumus <i>Gaussian Noise</i>	17
Rumus 2.3	Rumus Penskalaan Amplitudo	18
Rumus 2.4	Rumus Persamaan Transisi Keadaan	23
Rumus 2.5	Rumus <i>Weighted Cross-Entropy</i>	26
Rumus 2.6	Rumus Accuracy	28
Rumus 2.7	Rumus <i>Precision</i>	28
Rumus 2.8	Rumus <i>Recall</i>	29
Rumus 2.9	Rumus <i>F1-Score</i>	29
Rumus 2.10	Rumus Perhitungan Jumlah Parameter untuk lapisan konvolusi	30
Rumus 2.11	Rumus Perhitungan Jumlah Parameter untuk lapisan <i>fully</i> <i>connected</i>	30
Rumus 2.12	Rumus <i>Mean Absolute Error</i> (MAE)	31

DAFTAR KODE

Kode 4.1 Kode Segmentasi Data	48
Kode 4.2 Kode Normalisasi	49
Kode 4.3 Kode Augmentasi	52

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kantuk merupakan masalah universal yang berdampak negatif secara signifikan terhadap produktivitas dan yang lebih krusial, keselamatan nyawa manusia. Fenomena ini sering kali dianggap sepele, padahal konsekuensinya bisa sangat fatal terutama dalam konteks transportasi. Dampak nyata dari kondisi ini dapat terlihat jelas pada statistik kecelakaan lalu lintas di Indonesia. Menurut laporan resmi Badan Pusat Statistik, tercatat sebanyak 152.008 kasus kecelakaan yang terjadi sepanjang tahun 2023. Dari sekian banyak insiden tersebut, salah satu faktor utama yang konsisten menjadi penyebab kecelakaan adalah kelelahan atau kantuk yang dialami pengemudi saat berkendara [1]. Masalah ini diperparah oleh kegagalan penilaian mandiri (*self-assessment*) oleh para pengemudi. Sebuah studi menunjukkan fakta mengkhawatirkan bahwa 75% pengemudi sering kali salah menilai kondisi fisik mereka sendiri. Mereka merasa hanya "sedikit lelah" dan masih mampu mengemudi, padahal secara fisiologis mereka sebenarnya sudah berada dalam tingkat kantuk yang berbahaya dan berisiko tinggi mengalami *microsleep* [2].

Hingga saat ini, berbagai sistem deteksi kantuk telah dikembangkan untuk memitigasi risiko ini. Sebagian besar teknologi yang beredar di pasaran, seperti sistem berbasis kamera pemantau wajah atau sensor perilaku kendaraan, umumnya bersifat reaktif. Artinya, sistem tersebut baru mendeteksi bahaya dan memberikan peringatan setelah tanda-tanda fisik yang jelas muncul, seperti mata terpejam lama atau kendaraan keluar jalur [3]. Pendekatan reaktif tersebut dinilai kurang efektif dalam pencegahan dini, karena jeda waktu antara munculnya tanda fisik dan terjadinya kecelakaan sering kali terlalu singkat. Oleh karena itu, sangat diperlukan metode alternatif yang lebih akurat dan mampu mendeteksi

onset kantuk lebih awal sebelum manifestasi fisik yang berbahaya terjadi.

Untuk mendapatkan kemampuan deteksi dini tersebut, fokus penelitian kini bergeser dari pemantauan perilaku eksternal ke pemantauan sinyal fisiologis internal. Sinyal *electroencephalogram* (EEG) yang merekam aktivitas gelombang otak dan *electrooculogram* (EOG) yang merekam pola gerakan mata, diakui secara luas oleh komunitas ilmiah sebagai indikator kantuk yang paling andal dan objektif [4]. Keunggulan utama dari penggunaan kombinasi EEG dan EOG adalah kemampuannya untuk menangkap perubahan neurofisiologis yang mendasar seperti peningkatan gelombang Theta atau perlambatan gerakan mata yang terjadi jauh lebih cepat dibandingkan perubahan perilaku fisik yang terlihat kasat mata seperti menguap [3]. Validitas penggunaan modalitas ini juga didukung oleh fakta bahwa basis data standar emas (*gold standard*) untuk penelitian kantuk, seperti DROZY, secara spesifik menggunakan sinyal EEG dan EOG sebagai data utamanya untuk penentuan label kantuk [5].

Meskipun memiliki keunggulan dalam akurasi dan deteksi dini, analisis komputasi terhadap sinyal EEG dan EOG menghadirkan tantangan teknis tersendiri. Dalam beberapa tahun terakhir, model *deep learning* yang telah mapan, khususnya arsitektur berbasis Transformer, telah menunjukkan kinerja yang sangat kuat di banyak bidang pemrosesan data sekuensial. Namun, model-model canggih ini sering kali menghadapi tantangan *trade-off* komputasi yang berat saat harus memproses data sekuens yang sangat panjang, seperti rekaman sinyal EEG kontinu. Hal ini disebabkan oleh mekanisme *self-attention* pada arsitektur Transformer klasik yang memiliki kompleksitas kuadratik ($O(n^2)$). Kompleksitas ini membuat beban komputasi dan penggunaan memori meningkat secara drastis seiring bertambahnya panjang data yang diproses [6]. Mengingat sinyal EEG direkam dengan frekuensi tinggi dan dapat berlangsung lama, serta fakta bahwa transisi menuju kantuk terjadi secara bertahap dalam rentang waktu yang panjang, diperlukan sebuah model alternatif yang tidak hanya akurat, tetapi juga mampu secara efisien menangkap dependensi temporal jangka panjang

tanpa membebani sumber daya komputasi [7].

Tinjauan literatur mendalam mengidentifikasi adanya kesenjangan penelitian yang kritis untuk mengatasi hambatan tersebut. Arsitektur Mamba hadir sebagai solusi potensial berupa *State Space Model* (SSM) baru yang sangat efisien. Mamba menawarkan keunggulan kompleksitas linear $O(n)$ terhadap panjang sekuens, yang memungkinkannya mencapai *throughput* inferensi hingga 5× lebih cepat dibandingkan Transformer, menjadikannya kandidat ideal untuk pemrosesan sinyal biomedis yang panjang [6]. Model yang baru muncul pada akhir tahun 2023 ini telah menunjukkan keberhasilan empiris pada berbagai tugas pemrosesan EEG lainnya, seperti klasifikasi tahap tidur [7] dan *emotion recognition* [8]. Khusus untuk deteksi kantuk, baru tercatat satu studi awal yang menerapkan Mamba pada dataset SEED-VIG dengan capaian akurasi 83,24% [9]. Namun, eksplorasi terkait penerapan keunggulan arsitektur Mamba pada dataset standar DROZY saat ini masih sangat terbatas, meskipun DROZY telah menjadi benchmark yang paling banyak digunakan dalam studi kantuk dengan lebih dari 41 sitasi. Lebih spesifik lagi, analisis kombinasi sinyal EEG dan EOG menggunakan arsitektur Mamba juga belum banyak diteliti secara komprehensif, padahal kombinasi kedua modalitas ini telah terbukti superior dalam berbagai studi terdahulu [10].

Di samping kebutuhan eksplorasi arsitektur dan kombinasi modalitas tersebut, pendekatan pemodelan dalam mengukur tingkat kantuk juga menjadi isu krusial. Mayoritas penelitian sebelumnya menggunakan pendekatan klasifikasi diskrit sejak awal fase pelatihan, yang hingga kini menjadi standar umum dalam mendeteksi kondisi pengemudi. Namun, secara fisiologis, kantuk didefinisikan sebagai hilangnya efisiensi pemrosesan kortikal secara progresif, di mana transisi ini menyebabkan penurunan dan hilangnya fungsi kognitif secara bertahap [11]. Meskipun pemodelan klasifikasi diskrit mampu memberikan keputusan yang tegas, pemetaan proses yang sangat dinamis ini ke dalam label kategori secara langsung berpotensi menyamarkan informasi penting mengenai gradasi kelelahan

pengemudi.

Sebagai alternatif, pemodelan dengan keluaran kontinu memiliki keunggulan teoritis untuk menangkap resolusi deteksi secara gradual dan dinilai krusial untuk mendeteksi onset kantuk lebih awal [4]. Nilai prediksi kontinu dari model regresi ini nantinya tetap dipetakan menggunakan ambang batas (*threshold*) tertentu untuk menghasilkan keluaran klasifikasi biner pada tahap inferensi. Mengingat kedua pendekatan ini memiliki rasionalisasi masing-masing, sebuah studi perbandingan yang komprehensif antara model klasifikasi diskrit (biner) standar dengan model klasifikasi biner yang diturunkan dari hasil regresi sangat diperlukan. Perbandingan ini bertujuan untuk mengevaluasi secara objektif metode pelatihan mana yang menghasilkan prediksi akhir paling optimal, stabil, dan representatif dalam mendeteksi tingkat kantuk.

Berdasarkan seluruh uraian tersebut, penelitian ini bertujuan untuk mengisi kesenjangan yang ada dengan berfokus pada dua aspek utama: validasi pemanfaatan arsitektur Mamba untuk efisiensi pemodelan sekuens panjang, serta studi perbandingan antara model klasifikasi biner standar dengan model klasifikasi biner berbasis regresi. Sistem deteksi yang dikembangkan akan dievaluasi secara komprehensif pada dataset DROZY menggunakan kombinasi sinyal EEG dan EOG. Melalui studi komparatif ini, diharapkan penelitian ini tidak hanya mampu membuktikan efisiensi komputasi Mamba pada pemrosesan sinyal fisiologis, tetapi juga menemukan pendekatan pelatihan yang paling optimal. Hal ini diharapkan dapat mendorong pengembangan teknologi deteksi kantuk yang lebih responsif dan akurat demi menyelamatkan ribuan nyawa di jalan raya setiap tahunnya.

1.2 Rumusan Masalah

Adapun rumusan masalah dari penelitian ini adalah sebagai berikut:

1. Bagaimana merancang arsitektur model berbasis Mamba untuk deteksi tingkat kantuk berbasis sinyal fisiologis EEG dan EOG pada dataset

DROZY?

2. Bagaimana perbandingan kinerja antara model klasifikasi biner standar dengan model klasifikasi biner berbasis regresi dalam mendeteksi fase *Alert* dan *Drowsy*?

1.3 Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan arsitektur model berbasis Mamba untuk deteksi tingkat kantuk berbasis sinyal fisiologis EEG dan EOG pada dataset DROZY.
2. Mengevaluasi dan membandingkan kinerja antara model klasifikasi biner standar dengan model klasifikasi biner berbasis regresi dalam mendeteksi fase *Alert* dan *Drowsy*, guna menentukan pendekatan pelatihan yang paling optimal.

1.4 Batasan Masalah

Hal-hal berikut adalah batasan masalah dari penelitian ini:

1. Penelitian ini dibatasi pada pemanfaatan sinyal fisiologis berupa EEG dan EOG yang diekstraksi dari rekaman *Polysomnography* (PSG) pada dataset DROZY, tanpa melibatkan data visual atau pendekatan multimodal berbasis kamera.
2. Arsitektur *deep learning* yang dikembangkan berfokus pada penggunaan model Mamba sebagai pemodel sekuens temporal. Perbandingan dengan penelitian *state-of-the-art* (SOTA), khususnya yang menggunakan pendekatan multimodal, tidak dimaksudkan sebagai pembandingan akurasi secara langsung, melainkan sebagai konteks untuk mendiskusikan *trade-off* antara kompleksitas sistem dan efisiensi komputasi.
3. Penelitian ini berfokus pada pemrosesan dan analisis data yang telah direkam (*offline*). Penelitian ini tidak mencakup perancangan perangkat

keras, integrasi sensor tambahan, maupun implementasi sistem deteksi kantuk secara *real-time*.

1.5 Manfaat Penelitian

Manfaat dilaksanakannya penelitian ini adalah sebagai berikut.

1. Memberikan kontribusi ilmiah sebagai penerapan arsitektur Mamba pertama untuk deteksi kantuk menggunakan dataset DROZY.
2. Memvalidasi keunggulan efisiensi komputasi Mamba untuk sinyal gabungan EEG dan EOG.
3. Berkontribusi pada pengembangan teknologi deteksi kantuk yang lebih akurat dan dini.

1.6 Sistematika Penulisan

Gambaran umum isi laporan penelitian ini dijelaskan melalui sistematika penulisan berikut:

Bab I

Bab I berisikan tentang latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan laporan Tugas Akhir.

Bab II

Bab II membahas mengenai tinjauan pustaka dari penelitian terdahulu dan dasar teori yang berkaitan dengan penelitian ini.

Bab III

Bab III berisikan metode penelitian yang akan dilakukan. Berisikan penjelasan mengenai arsitektur Mamba sebagai metode untuk mendeteksi kantuk dengan sinyal EEG dan EOG.

Bab IV

Bab IV berisikan hasil dari pengembangan model yang dibuat berserta analisis dan evaluasi dari model yang dilakukan.

Bab V

Bab V berisikan kesimpulan hasil dari penelitian yang telah dilakukan serta saran akan pengembangan penelitian kedepannya.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Sebagai acuan untuk tugas akhir ini, bagian ini akan membahas beberapa penelitian sebelumnya. Penelitian-penelitian yang diulas ini memiliki kaitan dengan masalah dan solusi yang dibahas dalam laporan ini.

1. Penelitian oleh Siddhad et al. (2024) memperkenalkan DrowzEE-G-Mamba, yang merupakan studi *state-of-the-art* (SOTA) terdekat karena menjadi satu-satunya yang menerapkan arsitektur Mamba untuk deteksi kantuk. Penelitian ini menggunakan dataset SEED-VIG dan mencapai akurasi 83,24%. Metodenya memanfaatkan blok SS-Conv-SSM dan mekanisme 2D selective scan. Model ini dicatat cocok untuk implementasi real-time karena mampu mengurangi false positives dengan tetap mempertahankan akurasi [9].
2. Studi oleh Wang et al. (2025) mengusulkan EEGMamba, dasar model EEG pertama yang berbasis Mamba. Model ini menggunakan encoder Mamba *bidirectional* untuk memodelkan dependensi *spatiotemporal*. EEGMamba di *pre-train* menggunakan rekonstruksi *masked* EEG pada korpus data yang sangat besar dari lima dataset dan diberikan enam tugas dengan hirarki yang berbeda dijalankan sebagai evaluasi. Keenam tugas tersebut adalah *Emotion Recognition*, *Motor Imagery Classification*, *Sleep Staging*, *Seizure Detection*, *Imagined Speech Classification*, dan *Major Depressive Disorder Diagnosis*. Hasilnya menunjukkan bahwa EEGMamba mencapai kinerja SOTA di semua tugas. Secara khusus, model ini terbukti mengungguli model foundation berbasis Transformer, terutama dalam skenario *low-resource* [7].
3. Dalam penelitian BiT-MamSleep secara spesifik mengatasi masalah

klasifikasi tahap tidur, sebuah tugas yang sangat terkait dengan deteksi kantuk. Penelitian ini mengusulkan arsitektur baru yang menggabungkan Triple-Resolution CNN (TRCNN) untuk ekstraksi fitur multi-skala dengan mekanisme Bidirectional Mamba (BiMamba) untuk memodelkan dependensi temporal jangka panjang. Model ini juga menggunakan Focal Loss dan SMOTE untuk menangani masalah data tidak seimbang. Hasil evaluasi ekstensif pada empat dataset publik menunjukkan bahwa BiT-MamSleep secara signifikan mengungguli metode SOTA sebelumnya [12].

4. Penelitian oleh Cao et al. (2025) menjadi benchmark SOTA utama yang secara spesifik menggunakan dataset DROZY. Mereka mengusulkan metode deteksi kantuk multimodal yang mengintegrasikan sinyal fisiologis (EEG, ECG) dan gambar wajah. Model unggulan mereka, "multimodal feature coupled model" menggunakan ResNet18 untuk fitur gambar dan LSTM untuk fitur time-series (EEG/ECG). Keunikan model ini adalah fitur dari modalitas yang berbeda tidak hanya digabungkan, tetapi digunakan sebagai *mutual weights* untuk saling memengaruhi kontribusi akhir. Dengan pendekatan ini, mereka berhasil mencapai akurasi 98,41% dalam mengklasifikasikan 3 kategori kantuk (Alert, Low Vigilance, Drowsy) pada dataset DROZY [13].
5. *Review* oleh Li & Chung (2022) merangkum lanskap penelitian deteksi kantuk berbasis EEG secara komprehensif. Tinjauan ini mengonfirmasi bahwa sinyal EEG adalah indikator kantuk yang paling andal, mengungguli sinyal fisiologis lainnya. Hasil tinjauan ini menyoroti beberapa tantangan utama yang belum terpecahkan dalam bidang ini. Tantangan terbesar adalah kemampuan generalisasi, karena sinyal EEG sangat bervariasi antar individu, dan penelitian mengenai model yang lintas subjek masih sangat langka. Selain itu, disimpulkan bahwa untuk deteksi dini, model dengan output kontinu seperti regresi lebih baik daripada klasifikasi diskrit [4].

Ringkasan penelitian terdahulu dapat dilihat pada Tabel 2.1 berikut.

Tabel 2.1 Literasi Penelitian Terdahulu

No.	Judul	Masalah	Metode	Hasil
1.	DrowzEE-G-Mamba: Leveraging EEG and State Space Models for Driver Drowsiness Detection	Deteksi kantuk menggunakan EEG pada dataset SEED-VIG	Arsitektur Mamba (SS-Conv-SSM blocks)	Mencapai akurasi 83,24%.
2.	EEGMamba: An EEG foundation model with Mamba	Kebutuhan akan model dasar (foundation model) yang dapat digeneralisasi untuk berbagai tugas EEG (termasuk sleep staging)	Arsitektur Mamba sebagai foundation model yang di-pre-train pada 5 dataset publik	Model Mamba terbukti unggul (SOTA) di berbagai tugas EEG, menunjukkan keefektifannya untuk sinyal sekuensial EEG

No.	Judul	Masalah	Metode	Hasil
3.	BiT-MamSleep: Bidirectional Temporal Mamba for EEG Sleep Staging	Klasifikasi tahap tidur (sleep staging), sebuah tugas yang sangat mirip dengan deteksi kantuk	Bidirectional Mamba (BiMamba) dikombinasikan dengan TRCNN (Triple-Resolution CNN)	Secara signifikan mengungguli SOTA pada 4 dataset tahap tidur publik
4.	Optimized driver fatigue detection method using multimodal neural networks	Deteksi kantuk multimodal pada dataset DROZY	Multimodal Feature Coupled Model: Menggabungkan fitur EEG, ECG, dan gambar wajah, menggunakan ResNet18 dan LSTM	Mencapai akurasi 98,41% untuk klasifikasi 3 kategori (<i>Alert</i> , <i>Low Vigilance</i> , <i>Drowsy</i>) pada dataset DROZY

No.	Judul	Masalah	Metode	Hasil
5.	Electroencephalogram Based Approaches for Driver Drowsiness Detection and Management A Review	Merangkum berbagai pendekatan dan tantangan dalam deteksi kantuk berbasis EEG	Tinjauan Sistematis (Review)	Mengidentifikasi metode-metode yang ada (seperti CNN, LSTM) dan menyoroti tantangan generalisasi cross-subject

2.2 Dasar Teori

Berikut dipaparkan kerangka teori yang mendasari penelitian ini, khususnya terkait dengan metode klasifikasi kantuk menggunakan sinyal EEG dan EOG.

2.2.1 Klasifikasi Kantuk

Kantuk didefinisikan sebagai keadaan perantara antara terjaga dan tidur yang ditandai dengan penurunan kesadaran progresif serta keinginan kuat untuk tidur [11]. Dalam konteks klasifikasi, sangat penting untuk membedakan kantuk dari kelelahan. Karena kelelahan adalah keengganan melanjutkan tugas akibat kerja fisik atau pengalaman berkepanjangan yang membaik dengan istirahat, sedangkan istirahat dan ketidakaktifan justru dapat memperburuk rasa kantuk.

Klasifikasi kantuk bertujuan untuk memetakan kondisi penurunan kewaspadaan ini ke dalam tingkatan yang terukur secara objektif. Dalam studi simulasi berkendara, tingkatan kantuk ini sering dibagi menjadi skala kontinu yang terdiri dari lima kelas deskriptor, yaitu waspada, sedikit mengantuk, cukup mengantuk, sangat mengantuk, hingga ekstrem mengantuk. Validasi klasifikasi ini umumnya didasarkan pada observasi visual terhadap indikator

fisik subjek, seperti perubahan nada wajah, pergerakan mata atau penutupan kelopak mata yang lambat, serta perilaku spesifik lainnya [14].

2.2.2 Sinyal Fisiologis

Sinyal fisiologis, yang mencakup pengukuran aktivitas sistem saraf pusat dan sistem saraf otonom, dianggap sebagai indikator paling akurat untuk klasifikasi kantuk karena mampu mendefinisikan transisi fisiologis antara kondisi terjaga dan tidur secara presisi. Dalam konteks klasifikasi tingkat kantuk ditentukan dengan memantau penyimpangan parameter sinyal dari kondisi waspada, seperti peningkatan daya gelombang otak alpha dan theta pada EEG, penurunan detak jantung yang disertai peningkatan variabilitas detak jantung (HRV) pada ECG, serta peningkatan durasi dan frekuensi kedipan mata pada EOG [15].

2.2.2.1 Electroencephalogram (EEG)

Electroencephalogram (EEG) adalah metode pemantauan neurofisiologis yang merekam dinamika jaringan saraf otak, yang secara alami bersifat non-linear dan non-stasioner. Dalam konteks sistem deteksi kantuk pengemudi, sinyal EEG dianggap sebagai sumber data yang paling menjanjikan dan akurat dibandingkan metode lain seperti pelacakan mata atau perilaku kendaraan, karena kemampuannya untuk mendeteksi transisi keadaan internal otak secara langsung dari sadar menuju tidur. Sinyal ini sangat krusial karena fitur yang diekstraksi dari EEG menunjukkan pola yang konsisten baik untuk kantuk maupun kelelahan, menjadikannya dasar yang andal untuk membangun sistem peringatan dini bagi pengemudi [16].

Hubungan antara EEG dan klasifikasi tingkat kantuk didasarkan pada perubahan signifikan pola sinyal otak, di mana transisi menuju kelelahan umumnya berkorelasi dengan aktivitas pada pita frekuensi theta dan delta, serta fenomena atenuasi gelombang alpha yang membedakan kondisi terjaga santai

dengan permulaan tidur [17]. Selain perubahan pada pita frekuensi, klasifikasi kantuk juga dapat dideteksi melalui perubahan topologi konektivitas fungsional otak yang bergeser dari konfigurasi linear saat waspada menjadi konfigurasi berbentuk bintang saat mengantuk, terutama pada pita alpha dan beta [16].

2.2.2.2 Electrooculogram (EOG)

Electrooculogram (EOG) adalah sinyal elektrofisiologis yang dihasilkan oleh polarisasi bola mata, yang mencerminkan potensial istirahat antara kornea dan retina. Sinyal ini diukur melalui elektroda yang ditempatkan pada kulit di sekitar mata, di mana magnitudonya bervariasi sesuai dengan perpindahan bola mata dari posisi istirahatnya [18]. Secara teknis, turunan pertama dari sinyal EOG sebanding dengan kecepatan gerakan bola mata, yang memungkinkan analisis dinamika pergerakan mata secara detail. Amplitudo sinyal EOG umumnya berkisar antara 0,4 hingga 10 mV dengan frekuensi bervariasi dari 0,1 hingga 38 Hz, meskipun komponen frekuensi utamanya biasanya berada di bawah 10 Hz, sehingga EOG juga dapat dianggap sebagai bentuk elektromiografi dari otot-otot di sekitar mata [19].

Dalam konteks klasifikasi tingkat kantuk, EOG digunakan untuk mendeteksi transisi dari kondisi waspada ke mengantuk melalui perubahan pola pergerakan mata dan karakteristik kedipan. Saat seseorang dalam kondisi waspada, sinyal EOG menunjukkan dominasi *Rapid Eye Movements* (REM), sedangkan kondisi kantuk atau permulaan tidur ditandai dengan munculnya *Slow Eye Movements* (SEM) [20]. Selain itu, indikator fisiologis yang diekstraksi dari EOG, seperti *Average Blink Time* (ABT) dan *Wavelet Packet Barycenter Frequency* (WPBF), memiliki korelasi signifikan dengan tingkat kelelahan. Studi menunjukkan bahwa seiring meningkatnya kelelahan visual otot (*Muscular Visual Fatigue*), waktu kedipan rata-rata (ABT) akan meningkat secara signifikan sementara frekuensi barycenter (WPBF) akan menurun [19].

2.2.3 Pra-pemrosesan Sinyal

Pra-pemrosesan sinyal adalah serangkaian langkah awal yang sangat krusial dalam analisis EEG maupun EOG untuk membersihkan data mentah yang seringkali penuh dengan gangguan (*noise*). Proses ini bertujuan memisahkan aktivitas otak yang murni dari artefak yang tidak diinginkan. Sehingga karakteristik sinyal menjadi lebih jelas dan siap diolah. Hal ini mutlak diperlukan karena kualitas penerapan metode pra-pemrosesan akan sangat menentukan akurasi hasil analisis data selanjutnya. Jika sinyal masih kotor, kesimpulan yang diambil dari penelitian bisa menjadi tidak valid [21].

2.2.3.1 Segmentasi Sinyal

Segmentasi sinyal adalah proses mempartisi sinyal kontinu yang panjang menjadi bagian-bagian yang lebih kecil dan terpisah, yang sering disebut sebagai segmen atau epoch. Tujuannya adalah menyederhanakan analisis sinyal yang kompleks dan dinamis menjadi unit-unit yang lebih mudah dikelola, di mana karakteristik sinyal di dalam setiap segmen dianggap relatif stabil atau stasioner. Langkah ini krusial untuk memastikan bahwa fitur-fitur yang diekstraksi nantinya benar-benar merepresentasikan kondisi sesaat dari subjek yang diamati tanpa tercampur dengan perubahan kondisi di waktu lain [22].

Dalam proses ini, penentuan ukuran jendela (*window size*) atau durasi segmen menjadi parameter vital yang memengaruhi kualitas informasi yang diperoleh. Pemilihan durasi ini melibatkan kompromi teknis, yakni jendela yang terlalu panjang berisiko menyebabkan *information overload* atau hilangnya detail perubahan cepat karena tercampur dengan sinyal lain, sedangkan jendela yang terlalu pendek mungkin tidak memuat informasi yang cukup untuk menggambarkan pola sinyal secara utuh [23]. Oleh karena itu, ukuran segmen harus disesuaikan agar informasi relevan dapat terlokalisasi dengan tepat dalam rentang waktu tersebut.

2.2.3.2 Normalisasi Z-Score

Normalisasi Z-Score, atau standardisasi, adalah teknik pra-pemrosesan yang mentransformasikan fitur input dengan melakukan translasi dan penskalaan data terhadap nilai rata-rata (*mean*) dan variansinya. Dalam konteks analisis sinyal EEG dan pembelajaran mesin, metode ini tidak hanya bertujuan untuk menyeragamkan skala setiap fitur agar memiliki kontribusi yang seimbang dalam proses pelatihan, tetapi secara krusial berfungsi untuk mengeliminasi perbedaan karakteristik individu antar subjek atau sesi perekaman. Dengan menyamakan distribusi data ini, normalisasi Z-Score bertindak sebagai bentuk adaptasi domain yang efektif untuk memitigasi masalah pergeseran domain (*domain shift*) antara data pelatihan (*source*) dan data pengujian (*target*), sehingga meningkatkan kemampuan generalisasi model [24].

Secara matematis, transformasi Z-Score (Z) untuk sebuah himpunan data X dihitung menggunakan rumus berikut:

$$Z(X, \mu, \sigma) = \frac{X - \mu}{\sigma} \quad (\text{Rumus 2.1})$$

- X : Nilai data asli (fitur mentah).
- μ : Estimasi rata-rata (*mean*) dari distribusi data.
- σ : Estimasi standar deviasi (*standard deviation*) dari distribusi data.

2.2.4 Augmentasi Data Sinyal

Dalam pelatihan arsitektur *deep neural network*, salah satu tantangan utama yang sering dihadapi adalah kurangnya sampel data yang memadai untuk melatih model. Keterbatasan data ini dapat menyebabkan model mengalami *overfitting* dan pada akhirnya menghasilkan akurasi yang lebih rendah terhadap data yang belum pernah dilihat sebelumnya (*unseen samples*) [25].

Untuk mengatasi masalah tersebut, teknik augmentasi data diterapkan. Augmentasi data merujuk pada proses pembuatan sampel data pelatihan baru yang digunakan untuk meningkatkan kemampuan generalisasi model. Teknik ini

bertujuan untuk menghasilkan variasi sampel tambahan yang dapat membantu model pembelajaran mendalam dalam mengekstraksi fitur-fitur yang lebih diskriminatif [25].

2.2.4.1 *Gaussian Noise*

Salah satu metode augmentasi data yang sangat efektif untuk data deret waktu adalah injeksi *noise*. Metode ini mempertahankan karakteristik fundamental dari data asli sekaligus menambahkan *noise* alami ke seluruh kumpulan data. Secara matematis, proses injeksi *noise* dapat dideskripsikan melalui Persamaan Rumus 2.2:

$$x' = x + N(0, \sigma^2) \quad (\text{Rumus 2.2})$$

di mana x merepresentasikan data observasi asli, dan x' adalah data baru yang telah ditambahkan *noise*. Komponen $N(0, \sigma^2)$ menandakan *noise* acak yang dibangkitkan dari distribusi normal (*Gaussian distribution*) dengan nilai rata-rata (*mean*) sebesar 0 dan variansi σ^2 [26].

Melalui proses injeksi *noise* ini, arsitektur *deep learning* dapat mempelajari berbagai variasi pola fluktuasi tanpa mengubah sumbu waktu maupun tren utama dari data aslinya. Hal ini memungkinkan model untuk melakukan generalisasi dengan lebih baik terhadap data baru. Meskipun demikian, tingkat fluktuasi *noise* harus dikontrol dengan baik, karena penambahan *noise* yang terlalu ekstrem atau berlebihan justru dapat mengaburkan fitur penting dan mengganggu proses pembelajaran model [26].

2.2.4.2 *Amplitude Scaling*

Metode penskalaan (*scaling*) merupakan teknik augmentasi yang terdiri dari pengubahan magnitudo pada suatu langkah tertentu di dalam domain deret waktu. Ide utama dari teknik ini adalah untuk mempertahankan bentuk keseluruhan dari sinyal sembari mengubah nilai-nilainya. Melalui penskalaan, data baru yang dihasilkan akan mengalami perubahan pada rentang nilai, namun

tetap mempertahankan bentuk dari perubahannya.

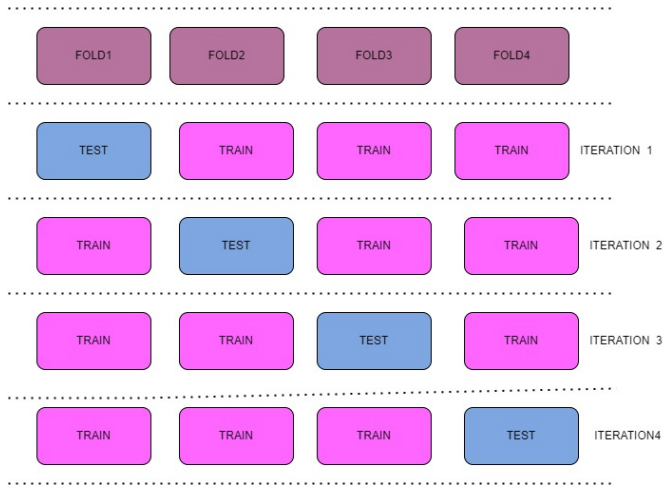
Dalam pemrosesan sinyal, teknik penskalaan amplitudo ini sering direpresentasikan sebagai penskalaan homogen (*homogeneous scaling*) yang dirumuskan sebagai:

$$x'(\alpha) = \{\alpha x_1, \dots, \alpha x_t, \dots, \alpha x_T\} \quad (\text{Rumus 2.3})$$

di mana parameter $\alpha > 0$ berfungsi untuk menentukan skala dari perubahan yang terjadi pada sinyal. Nilai skalar ini dapat ditentukan menggunakan distribusi *Gaussian* dengan nilai rata-rata (*mean*) 1 dan dengan σ sebagai *hyperparameter*, atau dapat pula ditentukan sebelumnya melalui sebuah daftar nilai yang sudah ditetapkan [27].

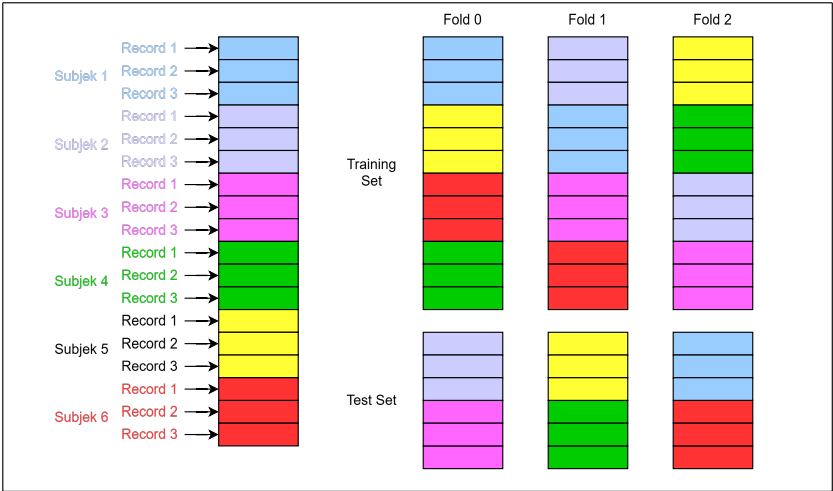
2.2.5 Cross Validation

Cross Validation adalah teknik pengembangan di *machine learning* yang digunakan untuk menjamin reliabilitas model dan mencegah terjadinya *overfitting*. Teknik yang paling umum diterapkan adalah *K-Fold Cross-Validation*, sebuah metode sistematis yang mempartisi dataset menjadi k himpunan bagian atau "folds" [28].



Gambar 2.1 Ilustrasi *K-Fold Cross Validation* [28]

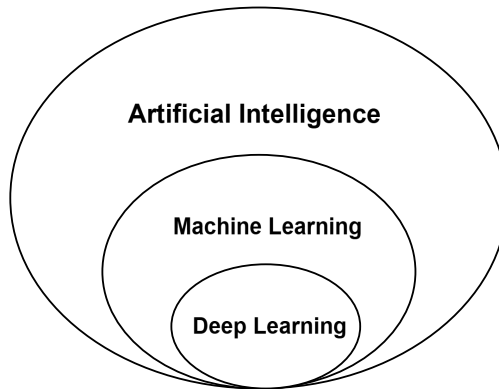
Gambar 2.1 memperlihatkan prosedur sistematis *K-Fold Cross-Validation* dengan membagi dataset menjadi k bagian. Seperti terlihat pada diagram, model dilatih secara berulang sebanyak k kali. Dalam setiap iterasi, $k - 1$ bagian (blok train) digunakan untuk pembelajaran, sementara satu bagian yang berbeda (blok test) digunakan untuk validasi secara bergantian. Metode ini memastikan setiap subset data berkontribusi sebagai data uji tepat satu kali, yang menghasilkan estimasi performa model yang lebih akurat dan tidak bias dibandingkan metode pemisahan data sederhana.



Gambar 2.2 Pembagian Dataset dengan teknik *Subject-wise cross-validation*

Untuk menangani karakteristik data yang kompleks, khususnya pada dataset medis yang memiliki banyak rekaman dari subjek yang sama, diperlukan teknik validasi yang mampu memperhitungkan ketergantungan data antar-subjek. Pada kasus seperti ini, penggunaan *K-Fold Cross-Validation* konvensional berpotensi menghasilkan evaluasi yang bias karena sampel dari satu subjek dapat muncul secara bersamaan pada data latih dan data uji. Oleh karena itu, pendekatan *subject-wise K-Fold Cross-Validation* digunakan, di mana seluruh rekaman yang berasal dari satu subjek ditempatkan secara eksklusif pada satu *fold* yang sama seperti yang terlihat pada gambar 2.2. Pendekatan berbasis subjek ini bertujuan untuk mencegah kebocoran data (*data leakage*) antar-subjek dan memastikan bahwa evaluasi performa model benar-benar mencerminkan kemampuan generalisasi terhadap subjek baru yang belum pernah dikenali sebelumnya [29].

2.2.6 Deep Learning



Gambar 2.3 Diagram lanskap *Artificial Intelligence*

Sebagaimana diilustrasikan dalam diagram lanskap *artificial intelligence* pada 2.3, *deep learning* bukanlah entitas yang berdiri sendiri, melainkan bagian spesifik dari hierarki sistem cerdas yang lebih besar. AI bertindak sebagai payung utama yang menaungi berbagai pendekatan, termasuk sistem berbasis aturan dan pengetahuan. Di dalam lingkup tersebut, terdapat Machine Learning sebagai subset yang berfokus pada kemampuan belajar dari data, dan *deep learning* hadir secara lebih spesifik di dalamnya sebagai metode yang terspesialisasi, menandakan posisinya sebagai evolusi teknik yang lebih mendalam dari pembelajaran mesin konvensional [30].

Deep learning adalah cabang dari *machine learning* yang berfokus pada cara komputer memahami data secara bertahap melalui lapisan-lapisan (*layers*) pemrosesan yang disusun secara berurutan. Istilah "deep" atau "dalam" di sini memiliki makna teknis yang sederhana, yaitu merujuk pada jumlah lapisan yang digunakan dalam model jaringan saraf untuk mengolah data, bukan pada kedalaman pemahaman yang bersifat filosofis. Secara prinsip, metode ini bekerja layaknya proses penyaringan informasi bertingkat, di mana data mentah dimasukkan dan diproses melewati banyak lapisan, sehingga di setiap lapisannya

data tersebut menjadi semakin "bersih" dan mudah dipahami oleh komputer untuk menyelesaikan tugas yang rumit [31].

Dalam *deep learning*, komputer memproses informasi secara bertingkat menggunakan model yang disebut jaringan saraf. Meskipun namanya terdengar biologis dan ide awalnya memang terinspirasi dari otak manusia, sebenarnya teknologi ini tidak meniru cara kerja otak kita sama sekali. Anggapan umum yang sering muncul bahwa *deep learning* bekerja persis seperti pikiran manusia adalah salah kaprah. Sederhananya, *deep learning* hanyalah sebuah kerangka matematika untuk mempelajari data [31].

2.2.7 State Space Models (SSM)

State Space Models (SSM) adalah kerangka kerja matematis berprinsip yang digunakan untuk memodelkan dan menganalisis deret waktu (*time series*), seperti data penjualan atau pola cuaca. Inti dari SSM adalah konsep keadaan laten (*latent state*), yang merupakan sekumpulan variabel tersembunyi yang berevolusi seiring waktu dan merekam informasi penting tentang deret waktu, seperti tren dan musiman. SSM didefinisikan oleh dua persamaan: Persamaan Transisi Keadaan, yang menjelaskan bagaimana keadaan laten berubah dari waktu ke waktu, dan Model Observasi, yang menjelaskan bagaimana data yang kita observasi dihasilkan dari keadaan laten tersebut. Kerangka ini disukai karena kemampuannya untuk dapat diinterpretasi dan efisien data, meskipun secara tradisional model ini diterapkan secara terpisah pada setiap deret waktu [32].

SSM didefinisikan oleh dua persamaan utama:

1. Persamaan Transisi Keadaan (*State-Transition Equation*): Menjelaskan dinamika transisi stokastik $p(l_t|l_{t-1})$ yang dengannya keadaan laten berevolusi seiring waktu [32]. Dalam konteks linier, ini seringkali berbentuk:

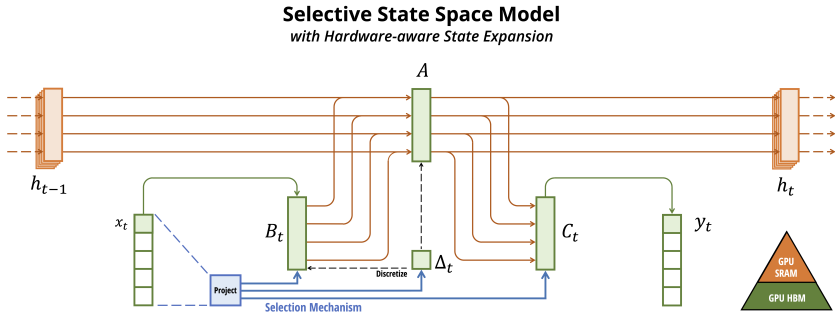
$$l_t = F_t l_{t-1} + g_t \epsilon_t \quad \epsilon_t \sim \mathcal{N}(0, 1) \quad (\text{Rumus 2.4})$$

Di mana F_t adalah matriks transisi deterministik, dan $g_t \in \epsilon_t$ adalah inovasi acak [32].

2. Model Observasi (*Observation Model*): Menentukan probabilitas bersyarat $p(z_t|l_t)$ dari observasi (z_t) yang diamati (data aktual) yang diberikan oleh keadaan laten [32].

2.2.8 Selective State Space Models

Selective State Space Models (Selective SSM) merupakan pengembangan dari *Structured State Space Models* (SSMs) konvensional yang dirancang untuk mengatasi kelemahan model *Linear Time Invariant* (LTI) dalam menyeleksi informasi yang relevan. Berbeda dengan SSM biasa yang parameter dinamikanya konstan terhadap waktu, Selective SSM memperkenalkan mekanisme seleksi yang membuat parameter model menjadi fungsi dari input itu sendiri (*input-dependent*).



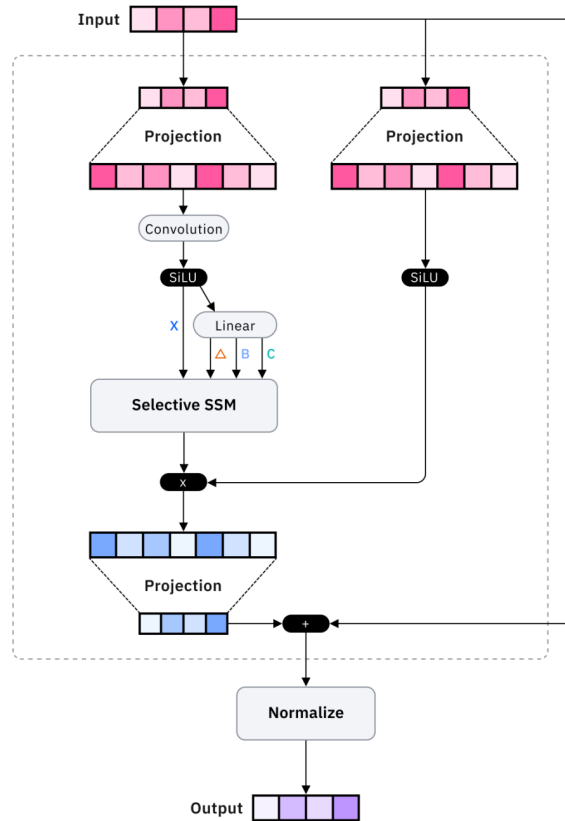
Gambar 2.4 Arsitektur Selective State Space Model [6]

Sebagaimana diilustrasikan pada Gambar 2.4, mekanisme seleksi bekerja dengan mengambil input x_t dan memproyeksikannya untuk menghasilkan parameter dinamis yang memungkinkan model secara selektif mempropagasikan atau melupakan informasi di sepanjang dimensi urutan. Karena perubahan

parameter ini mencegah penggunaan konvolusi yang efisien, Selective SSM menerapkan algoritma *hardware-aware*. Seperti yang ditunjukkan pada diagram memori di Gambar 2.4, algoritma ini menghitung model secara berulang (*recurrent*) menggunakan pemindaian (*scan*), namun tidak mematerialisasikan *state* yang diperluas (h_t) ke dalam memori HBM GPU yang lambat, melainkan memprosesnya di level memori SRAM yang lebih efisien untuk menghindari hambatan I/O [6].

2.2.9 Mamba

Mamba adalah arsitektur jaringan saraf tiruan baru yang dirancang untuk mengatasi inefisiensi komputasi arsitektur Transformer pada urutan data yang sangat panjang. Secara umum, Mamba adalah model sekuensial yang bekerja dalam waktu linear (*linear-time sequence modeling*), dibangun di atas komponen inti yang disebut *Selective State Space Models* (SSMs), tanpa menggunakan modul attention atau blok MLP (*Multi-Layer Perceptron*) yang biasanya ada di Transformer. Inovasi utamanya adalah membuat parameter SSM bergantung pada input data, yang memungkinkan model untuk secara adaptif memilih informasi mana yang harus disimpan dan mana yang harus dilupakan berdasarkan token saat ini. Kemampuan selektif ini secara efektif memungkinkan penalaran berbasis konten yang sebelumnya menjadi kelemahan model subkuadratik lainnya, menghasilkan inferensi yang cepat dan kinerja yang kompetitif di berbagai modalitas, termasuk bahasa [6].



Gambar 2.5 Diagram Blok Arsitektur Mamba [33]

Sebagaimana diperlihatkan pada Gambar 2.5, desain blok Mamba mengadopsi struktur *Gated MLP*. Proses dimulai dengan input yang diproyeksikan ke dimensi yang lebih besar (biasanya dengan faktor ekspansi $E = 2$) melalui lapisan *Linear Projection* ke dalam dua cabang paralel.

- Cabang Utama (Kiri): Data melewati konvolusi 1D pendek (*Convolution*) untuk menangkap ketergantungan lokal, kemudian melewati fungsi aktivasi SiLU. Setelah itu, sinyal masuk ke inti *Selective SSM*, di mana parameter B , C , dan Δ dihitung berdasarkan input (mekanisme seleksi) untuk memproses urutan secara rekursif.

- Cabang Gating (Kanan): Cabang ini berfungsi sebagai mekanisme gerbang (*gate*), di mana data diproyeksikan dan melewati fungsi aktivasi SiLU.

Output dari kedua cabang ini kemudian digabungkan melalui operasi perkalian *element-wise*, yang menggantikan mekanisme atensi tradisional. Hasil akhirnya diproyeksikan kembali ke dimensi model asli melalui lapisan *Linear Projection* terakhir dan ditambahkan dengan koneksi residual (simbol +) untuk stabilitas pelatihan. Struktur ini memungkinkan Mamba beroperasi sebagai *Recurrent Neural Network* (RNN) saat inferensi, memberikan kecepatan yang jauh lebih tinggi dibandingkan Transformer [33].

2.2.10 *Weighted Cross Entropy Loss*

Pada permasalahan klasifikasi dengan distribusi kelas yang tidak seimbang, penggunaan fungsi *loss* standar seperti *Cross-Entropy* cenderung menghasilkan model yang bias terhadap kelas mayoritas. Hal ini disebabkan karena *Cross-Entropy* memperlakukan setiap sampel secara setara tanpa mempertimbangkan frekuensi kemunculan masing-masing kelas, sehingga kesalahan pada kelas minoritas memberikan kontribusi yang relatif kecil terhadap total nilai *loss* selama proses pelatihan.

Untuk mengatasi permasalahan tersebut, *Weighted Cross-Entropy Loss* diperkenalkan sebagai pendekatan berbasis biaya (*cost-sensitive learning*) dengan memberikan bobot yang berbeda pada setiap kelas. Dengan cara ini, kesalahan prediksi pada kelas minoritas akan diberi penalti yang lebih besar, sehingga model terdorong untuk mempelajari representasi yang lebih seimbang antar kelas. Secara umum, *Weighted Cross-Entropy* dapat dirumuskan sebagai berikut.

$$\mathcal{L}_{WCE} = - \sum_{c=1}^C w_c y_c \log(\hat{y}_c), \quad (\text{Rumus 2.5})$$

di mana w_c merupakan bobot untuk kelas ke- c , y_c adalah label ground truth dalam bentuk *one-hot encoding*, dan $\hat{y}_c$ adalah probabilitas prediksi model untuk kelas ke- c .

Phan dan Yamamoto menunjukkan bahwa pemberian bobot pada *Cross-Entropy Loss* merupakan cara yang sederhana namun efektif untuk meningkatkan performa klasifikasi pada kelas minoritas tanpa memerlukan modifikasi arsitektur model atau teknik penyeimbangan data secara eksplisit. Hasil eksperimen mereka pada dataset dengan ketimpangan kelas yang tinggi menunjukkan bahwa *Weighted Cross-Entropy* mampu meningkatkan performa kelas minoritas secara signifikan, sementara performa keseluruhan tetap terjaga dibandingkan dengan *Cross-Entropy* standar [34].

2.2.11 *Confusion Matrix*

Confusion matrix adalah teknik evaluasi yang mendetail dan komprehensif untuk mengukur kinerja model klasifikasi dalam pembelajaran mesin (*supervised learning*) [35]. Konsep ini, yang awalnya dikembangkan oleh Karl Pearson pada tahun 1904 sebagai tabel kontingensi, direpresentasikan sebagai matriks persegi berukuran $N \times N$, di mana N mewakili jumlah kelas output. Matriks ini memvisualisasikan perbandingan antara label kelas sebenarnya (*ground truth*) dengan label yang diprediksi oleh model. Dalam struktur ini, elemen-elemen pada diagonal utama menunjukkan jumlah prediksi yang benar, sementara elemen di luar diagonal merepresentasikan kesalahan klasifikasi atau *misclassification*. Keunggulan utama dari *confusion matrix* adalah kemampuannya untuk memberikan wawasan mendalam mengenai perilaku internal pengklasifikasi (*classifier*), mengungkap kelemahan model, serta membedakan jenis kesalahan yang terjadi, seperti *Type I error* (False Positive) dan *Type II error* (False Negative) [35].

		Predicted Values	
		Positive	Negative
Actual Values	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

Gambar 2.6 Confusion Matrix

Gambar 2.6 menyajikan visualisasi confusion matrix yang memetakan hasil prediksi model terhadap kelas aktualnya. Informasi yang terkandung dalam matriks ini selanjutnya digunakan sebagai dasar perhitungan untuk berbagai metrik evaluasi kinerja berikut:

1. *Accuracy*: Mengukur rasio total prediksi yang benar terhadap keseluruhan sampel data [36].

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (\text{Rumus 2.6})$$

2. *Precision*: Mengukur proporsi prediksi positif yang benar-benar positif. Metrik ini dihitung dengan membagi elemen diagonal dengan jumlah total elemen pada kolom terkait [35].

$$Precision = \frac{TP}{TP + FP} \quad (\text{Rumus 2.7})$$

3. *Recall*: Mengukur proporsi kasus positif aktual yang berhasil diidentifikasi dengan benar oleh model. Metrik ini dihitung dengan membagi elemen diagonal dengan jumlah total elemen pada baris terkait [35].

$$Recall = \frac{TP}{TP + FN} \quad (\text{Rumus 2.8})$$

4. *F1-Score*: Merupakan rata-rata harmonik (*harmonic mean*) dari presisi dan *recall*. Metrik ini memberikan keseimbangan antara kedua nilai tersebut dan lebih cocok digunakan pada data yang memiliki ketidakseimbangan kelas [35].

$$F1\text{-score} = 2 \times \frac{Precision \times Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \quad (\text{Rumus 2.9})$$

2.2.12 Evaluasi Kompleksitas Komputasi

Efisiensi komputasi adalah ukuran seberapa hemat dan cepat sebuah algoritma bekerja, bukan hanya seberapa pintar algoritma itu menebak dengan benar. Evaluasi ini melibatkan analisis kompleksitas waktu, seperti durasi pelatihan dan prediksi, serta kompleksitas ruang yang mengukur konsumsi memori selama operasional model. Selain itu, efisiensi juga dinilai dari optimalisasi pemanfaatan perangkat keras (CPU dan GPU) untuk memastikan algoritma dapat berjalan efektif, terutama dalam lingkungan dengan sumber daya terbatas [37].

Evaluasi kompleksitas komputasi bertujuan untuk menilai biaya komputasi yang dibutuhkan oleh suatu model pembelajaran mesin dalam proses pelatihan maupun inferensi. Evaluasi ini penting terutama pada pemodelan sekuens panjang sinyal fisiologis, di mana kompleksitas model dapat meningkat secara signifikan. Beberapa metrik yang umum digunakan untuk mengevaluasi efisiensi komputasi meliputi jumlah parameter dan GFLOPs.

2.2.12.1 Jumlah Parameter

Jumlah parameter merujuk kepada total variabel internal yang ada pada sebuah model, terutama bobot (*weight*) dan bias yang nilainya dipelajari dan diperbarui selama proses pelatihan untuk meminimalisir *loss function*. Jumlah parameter sering digunakan sebagai indikator utama untuk mengukur kapasitas memori model serta kompleksitas komputasi yang diperlukan, di mana peningkatan jumlah parameter biasanya berkorelasi dengan peningkatan performa model namun juga meningkatkan biaya pelatihan dan latensi inferensi [38].

Perhitungan jumlah parameter bergantung pada jenis lapisan (layer) yang digunakan dalam arsitektur jaringan. Untuk lapisan konvolusi (convolutional layer), jumlah parameter dihitung berdasarkan ukuran kernel, jumlah filter input, dan jumlah filter output dengan rumus sebagai berikut [39].

$$\text{Params} = K \cdot C^2 \cdot D + K \quad (\text{Rumus 2.10})$$

Di mana: K adalah jumlah filter, C adalah ukuran spasial kernel, dan D adalah kedalaman atau jumlah *channel input*.

Sementara itu, untuk lapisan *fully connected*, jumlah parameter adalah hasil perkalian antara ukuran input (I) dan ukuran output (O) ditambah dengan bias [39].

$$\text{Params} = (I \cdot O) + O \quad (\text{Rumus 2.11})$$

2.2.12.2 GFLOPs

FLOPs (*Floating Point Operations*) adalah satuan metrik standar yang digunakan untuk mengukur kompleksitas komputasi atau "biaya" operasional dari sebuah model *Deep Learning*. Secara teknis, metrik ini menghitung total operasi matematika dasar, khususnya perkalian dan penjumlahan pada bilangan desimal yang harus dilakukan model saat melakukan inferensi atau selama pelatihan [39].

Karena total operasi pada model yang kompleks sering kali mencapai angka miliaran, penggunaan satuan dasar FLOPs menjadi tidak praktis, sehingga representasinya sering dikonversi ke dalam skala yang lebih besar seperti GFLOPs (*Giga Floating Point Operations*). GFLOPs merepresentasikan jumlah miliar operasi *floating-point* yang dieksekusi, yang berfungsi sebagai indikator utama untuk mengukur kompleksitas komputasi dan efisiensi model pada berbagai platform perangkat keras [40]. Dalam evaluasi model, nilai GFLOPs yang lebih rendah menandakan model yang lebih ringan dan membutuhkan daya komputasi yang lebih sedikit, sehingga lebih ideal untuk penerapan *real-time* atau pada perangkat dengan sumber daya terbatas.

2.2.13 Mean Absolute Error (MAE)

Dalam konteks pemodelan regresi, pengukuran kesalahan rata-rata (*average error*) merupakan hal yang esensial untuk mengevaluasi kinerja model. *Mean Absolute Error* (MAE) atau yang juga dikenal sebagai *L1 Loss*, merupakan ukuran kesalahan rata-rata yang paling alami (*natural*) dan tidak ambigu (*unambiguous*) jika dibandingkan dengan metrik lain seperti *Root Mean Square Error* (RMSE) [41].

Perhitungan MAE secara konseptual sangat sederhana dan transparan, yaitu dengan menjumlahkan magnitudo atau nilai mutlak (*absolute value*) dari setiap kesalahan prediksi individual, kemudian membaginya dengan jumlah total sampel. Secara matematis, formulasi MAE dapat dirumuskan sebagai berikut:

$$MAE = n^{-1} \sum_{j=1}^n |e_j| \quad (\text{Rumus 2.12})$$

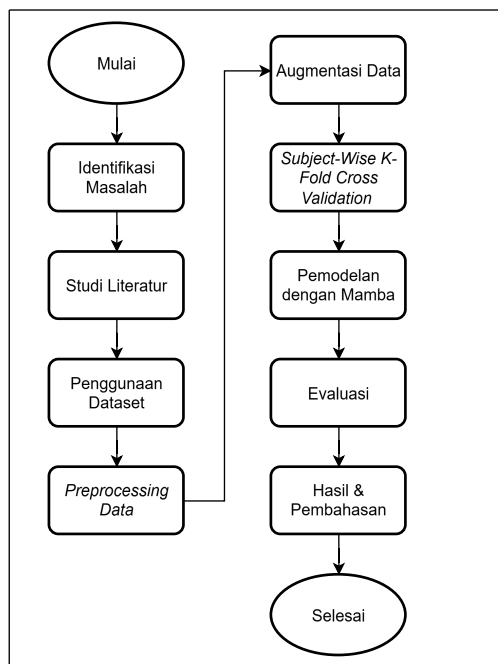
di mana n adalah jumlah total observasi, dan e_j adalah nilai kesalahan (*error*) individual pada sampel ke- j . Nilai kesalahan e_j sendiri secara spesifik didefinisikan sebagai selisih antara nilai estimasi atau prediksi model (P_j) dengan nilai observasi aktual (O_j), yang direpresentasikan dengan $e_j = P_j - O_j$ [41].

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Keseluruhan proses penelitian dalam tugas akhir ini dirancang secara sistematis dan dituangkan ke dalam sebuah alur penelitian. Alur tersebut selanjutnya diilustrasikan melalui diagram alir yang disajikan pada Gambar 3.1.



Gambar 3.1 Alur Penelitian

3.2 Penjabaran Langkah Penelitian

Untuk memberikan pemahaman yang jelas mengenai metode yang digunakan, langkah-langkah dalam penelitian ini akan dijabarkan secara lebih terperinci.

3.2.1 Identifikasi Masalah

Tahap pertama dalam alur penelitian ini adalah identifikasi masalah, yang dilakukan untuk memetakan konteks dan urgensi dari penelitian. Proses ini diawali dengan analisis terhadap permasalahan di dunia nyata, yaitu tingginya risiko keselamatan yang disebabkan oleh kantuk saat berkendara. Analisis ini divalidasi dengan meninjau data statistik kecelakaan, seperti laporan BPS, serta studi pendukung yang mengonfirmasi bahwa pengemudi cenderung gagal menilai tingkat kantuk mereka secara objektif.

Setelah urgensi masalah teridentifikasi, proses dilanjutkan dengan menganalisis solusi sistem deteksi kantuk yang ada saat ini. Hasil analisis menemukan bahwa sebagian besar sistem komersial, seperti yang berbasis kamera, memiliki kelemahan fundamental, yaitu bersifat reaktif. Sistem tersebut baru memberikan peringatan setelah tanda-tanda fisik kantuk termanifestasi. Identifikasi kelemahan sistem reaktif ini menegaskan kebutuhan akan pendekatan yang lebih proaktif, sehingga penelitian ini difokuskan pada penggunaan sinyal fisiologis (EEG dan EOG) sebagai domain utama yang akan dieksplorasi.

3.2.2 Studi Literatur

Setelah masalah utama teridentifikasi, langkah selanjutnya dalam alur penelitian adalah melakukan studi literatur secara sistematis. Proses ini bertujuan untuk memvalidasi *research gap* yang telah diidentifikasi pada Bab 1, mengumpulkan landasan teori yang relevan, serta memetakan metode *state-of-the-art* (SOTA) yang akan digunakan sebagai pembanding dalam penelitian ini.

Studi literatur difokuskan pada tiga area utama. Pertama, meninjau arsitektur *State Space Model* (SSM) dan Mamba, serta studi-studi terbaru yang menerapkannya pada sinyal EEG seperti klasifikasi tahap tidur. Kedua, mengidentifikasi metode SOTA yang ada saat ini khusus pada dataset DROZY, seperti model multimodal dari Cao et al. [13], untuk menetapkan *benchmark*

performa. Ketiga, studi ini secara khusus memvalidasi kesenjangan metodologis yang ditemukan di Bab 1, yaitu belum ditemukannya penerapan Mamba pada DROZY.

3.2.3 Penggunaan Dataset

Penelitian ini menggunakan dataset publik "The ULg Multimodality Drowsiness Database" (DROZY). Dataset ini dirancang khusus untuk penelitian pemantauan kantuk dan berisi rekaman dari 14 subjek sehat (3 pria, 11 wanita). Proses pengumpulan data dilakukan dalam lingkungan laboratorium terkontrol di mana setiap subjek menjalani tiga sesi *Psychomotor Vigilance Test* (PVT) berdurasi kurang lebih 10 menit. Sesi-sesi ini dilakukan dalam kondisi kekurangan tidur yang semakin meningkat, dengan total durasi kekurangan tidur mencapai 28 hingga 30 jam, untuk menginduksi kantuk secara alami.

Sesuai dengan batasan masalah penelitian, data utama yang digunakan terdiri dari label kantuk subjektif serta sinyal fisiologis (EEG & EOG) yang diekstrak dari rekaman *Polysomnography* (PSG). Sinyal PSG dalam dataset ini direkam pada frekuensi 512 Hz dan mencakup 5 channel EEG (Fz, Cz, Pz, C3, C4) serta 2 channel EOG (vertikal dan horizontal). Sebagai *ground truth* untuk melatih model, penelitian ini menggunakan Karolinska Sleepiness Scale (KSS), sebuah laporan subjektif 9-level (1=Sangat Segar, 9=Sangat Mengantuk) yang dicatat oleh subjek di awal setiap sesi PVT. Meskipun idealnya terdapat 42 rekaman (14 subjek $\times$ 3 sesi), beberapa data hilang dan satu sesi tidak dilakukan (7-1) akibat insiden teknis. Oleh karena itu, total bahan penelitian yang digunakan dalam penelitian ini adalah 36 set data (file .edf) [5].

Terdapat 9 level pada skor KSS asli. Sejalan dengan pendekatan pada studi terbaru yang juga menggunakan dataset DROZY, tujuan utama dalam implementasi sistem adalah mendeteksi status mengantuk secara tegas, sehingga tidak diperlukan pembedaan yang terlalu spesifik pada tingkat kewaspadaan menengah [42]. Berdasarkan rasionalisasi tersebut, pemrosesan label KSS pada

penelitian ini dilakukan dalam dua skenario utama untuk mengakomodasi studi perbandingan:

1. Skenario Klasifikasi (*Train Baseline*): Untuk menyederhanakan tingkat kewaspadaan menengah menjadi keputusan akhir yang tegas, label skor KSS aktual dikelompokkan secara langsung menjadi klasifikasi biner (2 kelas) untuk pelatihan model *baseline* klasifikasi [42]. Pengelompokan dasar ini menggunakan batas kategori KSS 5, di mana kondisi *Alert* secara mutlak didefinisikan untuk skor KSS diskrit 1-5, dan kondisi *Drowsy* untuk skor KSS 6-9.
2. Skenario Regresi (*Train & Inferensi*): Nilai KSS asli (1-9) dinormalisasi menjadi rentang kontinu $[0, 1]$ dengan cara membaginya menggunakan nilai maksimum KSS (9.0). Transformasi target ke rentang $[0, 1]$ ini merupakan praktik esensial pada pemodelan regresi berbasis jaringan saraf untuk mencegah munculnya gradien kesalahan (*exploding gradients*) yang ekstrem, sehingga algoritma optimasi dapat mencapai konvergensi dengan lebih cepat dan stabil. Pada tahap inferensi, setelah nilai prediksi dikembalikan ke skala asli (1-9), pemetaan ke kelas biner tidak menggunakan ambang batas bilangan bulat. Pemetaan dilakukan menggunakan nilai tengah sebesar 5.5. Keputusan ini diambil untuk mengakomodasi pembulatan matematis secara adil, di mana nilai prediksi ≤ 5.5 akan dikategorikan sebagai *Alert* (mewakili rentang KSS 1-5), dan nilai > 5.5 dikategorikan sebagai *Drowsy* (mewakili rentang KSS 6-9).

3.2.4 *Subject Wise K-Fold Cross Validation*

Setelah tahap *preprocessing*, validasi model dilakukan menggunakan *subject-wise K-Fold Cross-Validation* dengan nilai $k = 5$. Pada skema ini, pembagian data dilakukan berdasarkan subjek, sehingga seluruh sampel dari satu subjek hanya muncul pada satu fold yang sama. Pada setiap iterasi, satu fold digunakan sebagai data uji dan $k - 1$ fold lainnya sebagai data latih secara

bergiliran. Pendekatan ini digunakan untuk mencegah kebocoran data antar subjek serta mengevaluasi kemampuan generalisasi model terhadap subjek yang belum pernah dilihat sebelumnya.

3.2.5 *Preprocessing Data*

Tahap prapemrosesan data bertujuan untuk mengubah data sinyal PSG mentah menjadi format sekuensial yang terstruktur dan siap digunakan oleh model Mamba. Proses prapemrosesan ini terdiri dari dua langkah utama, di mana langkah pertama adalah segmentasi sinyal menggunakan teknik *sliding window*. Sinyal mentah 7 kanal (5 EEG: Fz, Cz, C3, C4, Pz, dan 2 EOG: EOG-V, EOG-H) yang direkam secara kontinu pada frekuensi 512 Hz dipotong dengan jendela pengamatan (*window size*) berdurasi 30 detik. Pergeseran antar-jendela (*stride*) ditetapkan selama 10 detik, sehingga secara otomatis menghasilkan area tumpang tindih (*overlap*) sebesar 20 detik (sekitar 66%) antar-segmen yang berdekatan. Mengacu pada frekuensi *sampling* tersebut, setiap segmen 30 detik akan menghasilkan tensor berukuran $[7 \times 15.360]$ titik data yang diperlakukan sebagai satu sampel data individual. Pemilihan durasi 30 detik ini didasarkan pada metodologi yang digunakan oleh Zhao et al. (2024), yang merupakan penelitian *state-of-the-art* untuk implementasi arsitektur Mamba pada kasus klasifikasi tingkat kantuk. Dalam studi tersebut, jendela waktu 30 detik dipilih karena merepresentasikan durasi standar satu *epoch* sesuai pedoman klinis AASM (*American Academy of Sleep Medicine*) untuk analisis tahapan tidur. Hal ini sangat penting karena memungkinkan model untuk menangkap dependensi temporal jangka panjang serta transisi bertahap antar-fase kesadaran yang sangat krusial [12].

Selain tahapan segmentasi, dilakukan juga proses normalisasi sinyal. Berbeda dengan pendekatan standar yang umumnya melakukan normalisasi secara global pada setiap saluran (*channel*) melintasi seluruh data, penelitian ini menerapkan teknik normalisasi berbasis subjek (*subject-specific normalization*).

Sinyal fisiologis seperti EEG dan EOG diketahui memiliki variabilitas antar-subjek (*inter-subject variability*) yang tinggi, di mana amplitudo dan karakteristik dasar gelombang otak dapat berbeda secara signifikan antar individu, bahkan pada tingkat kewaspadaan yang sama.

Oleh karena itu, normalisasi *Z-score* diaplikasikan secara terpisah untuk setiap individu. Proses ini dilakukan dengan menghitung nilai rata-rata dan simpangan baku secara spesifik dari keseluruhan data rekaman milik satu subjek tertentu. Nilai amplitudo sinyal dari subjek tersebut kemudian dikurangi dengan rata-ratanya sendiri dan dibagi dengan simpangan bakunya masing-masing. Alur komputasi ini diulangi untuk setiap subjek yang ada di dalam dataset. Melalui pendekatan ini, data sinyal dari setiap pengemudi akan memiliki skala yang seragam, sehingga model Mamba dapat lebih fokus dalam mengekstraksi pola penurunan kewaspadaan yang relatif dari masing-masing individu tanpa terdistorsi oleh perbedaan karakteristik bawaan antar subjek.

3.2.6 Augmentasi Data

Proses augmentasi pada penelitian ini dilakukan secara dinamis (*on-the-fly*) pada setiap *epoch* pelatihan. Terdapat dua teknik transformasi yang diaplikasikan secara independen, di mana masing-masing teknik memiliki probabilitas sebesar 50% untuk diterapkan pada setiap segmen sampel sinyal. Penggunaan parameter yang kecil (*mild augmentation*) ini dirancang secara spesifik untuk sinyal biologis, guna memastikan bahwa penambahan variasi tidak merusak informasi penting yang terkandung di dalamnya. Kedua teknik tersebut adalah:

1. *Gaussian Noise*: Penyuntikan derau acak berdistribusi normal ke dalam sinyal untuk menyimulasikan gangguan pada sensor maupun variasi biologis minor. Parameter derau yang ditambahkan memiliki standar deviasi (*STD*) sebesar 0.01.
2. *Amplitude Scaling*: Pengubahan skala amplitudo sinyal secara acak untuk menyimulasikan variabilitas kekuatan sinyal antar-subjek maupun antar-

sesi perekaman. Faktor skala ditarik secara acak dari rentang 0.9 hingga 1.1.

Melalui penerapan probabilistik dari kedua teknik ini, model Mamba dilatih untuk mengekstrak representasi fitur temporal yang lebih tangguh (*robust*) tanpa kehilangan pola fundamental dari sinyal kantuk itu sendiri.

3.2.7 Pemodelan dengan Mamba

Sesuai dengan alur penelitian, setelah data mentah disiapkan melalui tahap *preprocessing*, langkah selanjutnya adalah tahap pemodelan dengan Mamba. Tahap ini merupakan inti dari eksperimen penelitian, di mana model *deep learning* yang diusulkan akan dibangun, dilatih, dan divalidasi. Pada bagian ini, akan dijelaskan proses penggunaan arsitektur Mamba untuk memproses sekuens sinyal EEG dan EOG yang telah disegmentasi. Tujuan utama dari model ini adalah untuk mempelajari pola-pola temporal kompleks di dalam sinyal tersebut yang berkorelasi dengan 2 kategori tingkat kantuk yang didasari dari skor KSS.

3.2.8 Evaluasi

Evaluasi kinerja model dalam penelitian ini dilakukan secara menyeluruh mencakup aspek ketepatan prediksi dan efisiensi komputasi. Untuk mengukur performa prediksi pada tahap pelatihan regresi, digunakan metrik *Mean Absolute Error* (MAE). Sementara itu, pada tugas klasifikasi biner di tahap inferensi akhir, digunakan metrik Akurasi, Presisi, *Recall*, dan *F1-Score*. Akurasi mengukur persentase total prediksi yang benar, Presisi menghitung proporsi prediksi positif yang relevan, sedangkan *Recall* mengukur kemampuan model mendeteksi seluruh sampel positif (yaitu kelas *Drowsy*). *F1-Score* kemudian dihitung sebagai rata-rata harmonik antara Presisi dan *Recall*. Di samping itu, efisiensi komputasi model dievaluasi melalui pengukuran GFLOPs untuk mengestimasi kompleksitas operasi dan jumlah parameter untuk mengetahui kapasitas spasial model. Terakhir, *Confusion Matrix* digunakan untuk memvisualisasikan hasil

klasifikasi biner agar distribusi kesalahan antara kelas *Alert* dan *Drowsy* dapat dianalisis secara detail.

3.3 Lingkungan Pengembangan Tugas Akhir

Pelaksanaan penelitian ini membutuhkan lingkungan pengembangan yang memadai, baik dari sisi komputasi fisik maupun perangkat lunak, guna mendukung kelancaran proses pra-pemrosesan data sinyal fisiologis hingga pelatihan arsitektur *deep learning* yang kompleks. Lingkungan pengembangan ini dibagi menjadi dua kategori utama, yaitu perangkat keras dan perangkat lunak.

3.3.1 Perangkat Keras (*Hardware*)

Perangkat keras utama yang digunakan dalam penelitian ini adalah satu unit laptop Asus Vivobook M415DA yang berfungsi sebagai stasiun kerja utama untuk penulisan kode, analisis data tahap awal, serta manajemen repositori proyek. Laptop yang digunakan beroperasi dengan sistem operasi Windows 11, serta ditenagai oleh prosesor AMD Ryzen 3-3250U, memori RAM sebesar 8 GB, dan media penyimpanan SSD berkapasitas 512 GB.

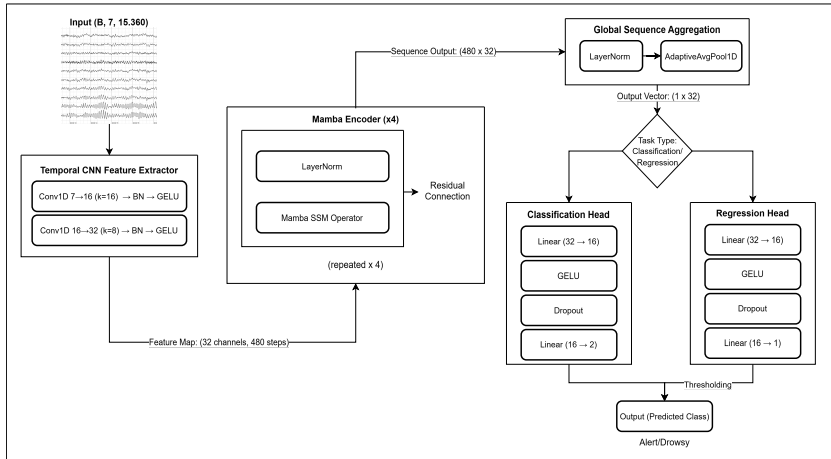
Meskipun arsitektur Mamba dirancang dengan tingkat efisiensi komputasi yang tinggi, implementasi dan kompilasi library intinya saat ini sangat dioptimalkan dan lebih stabil untuk berjalan pada ekosistem sistem operasi berbasis Linux. Mengingat perangkat keras lokal beroperasi menggunakan Windows 11, laptop ini utamanya berperan sebagai antarmuka untuk pengembangan skrip awal. Untuk menghindari kendala kompatibilitas sistem operasi dan mempermudah proses kompilasi pustaka CUDA, eksekusi pelatihan model secara penuh dilakukan melalui layanan *cloud computing* yang telah menyediakan lingkungan Linux secara *native* beserta dukungan unit pemrosesan grafis (GPU).

3.3.2 Perangkat Lunak (*Software*)

Selain perangkat keras, penelitian ini juga sangat bergantung pada susunan perangkat lunak (*software stack*) yang saling terintegrasi. Beberapa perangkat lunak utama yang digunakan meliputi:

1. Bahasa Pemrograman Python: Digunakan sebagai bahasa pemrograman utama karena kapabilitasnya yang sangat luas dan dukungan ekosistem pustaka (*library*) yang matang untuk keperluan *machine learning* dan pemrosesan sinyal.
2. PyTorch: Berfungsi sebagai *framework deep learning* utama untuk membangun, melatih, dan mengevaluasi arsitektur Mamba. PyTorch dipilih karena fleksibilitasnya dalam komputasi tensor dinamis berbasis GPU.
3. Google Colab: Digunakan sebagai platform komputasi awan berbasis *Jupyter Notebook* untuk mengeksekusi kode secara interaktif dan mendapatkan akses ke sumber daya GPU secara efisien tanpa terkendala spesifikasi perangkat keras lokal.
4. Visual Studio Code (VS Code): Berfungsi sebagai *Integrated Development Environment* (IDE) lokal untuk menulis, merapikan, dan menguji skrip pemrograman.
5. Git dan GitHub: Digunakan sebagai (*version control system*) untuk merekam jejak perubahan kode, serta menyimpan cadangan repositori proyek secara aman.
6. Weights & Biases: Digunakan sebagai pustaka pendukung untuk memantau, mencatat metrik evaluasi (seperti MAE dan *Loss*), serta mengelola parameter selama proses eksperimen dan pelatihan model berlangsung.

3.4 Metode Pengembangan



Gambar 3.2 Pemodelan dengan Mamba

Arsitektur pemodelan, sebagaimana diilustrasikan pada Gambar 3.2, dirancang untuk memproses segmen data yang telah dipersiapkan pada tahap *preprocessing*. Model menerima masukan data 30 detik dengan dimensi Input (B, 7, 15.360), yang merepresentasikan 7 *channel* (5 EEG + 2 EOG). Masukan ini pertama kali diproses oleh *Temporal CNN Feature Extractor* yang terdiri dari dua lapisan *Conv1D* dengan konfigurasi filter $7 \rightarrow 16$ ($k = 16$) dan $16 \rightarrow 32$ ($k = 8$), yang masing-masing diikuti oleh *Batch Normalization* (BN) dan fungsi aktivasi GELU. Proses ini menghasilkan *Feature Map* dengan dimensi (32 *channels*, 480 *steps*) yang mereduksi panjang sekuens namun memperkaya representasi fitur temporal lokal.

Inti dari arsitektur ini adalah *Mamba Encoder (x4 block)*, yang terdiri dari 4 blok Mamba yang ditumpuk secara berulang. Setiap blok terdiri dari *LayerNorm* dan *Mamba SSM Operator* dengan tambahan *Residual Connection* untuk menjaga stabilitas aliran gradien. Pemilihan arsitektur *State Space Model* (SSM) ini didasarkan pada efisiensi komputasi linear ($O(n)$) yang memungkinkan

model secara selektif menentukan informasi temporal jangka panjang pada sinyal EEG dan EOG yang harus dipertahankan atau diabaikan melalui mekanisme *selective state spaces* [6].

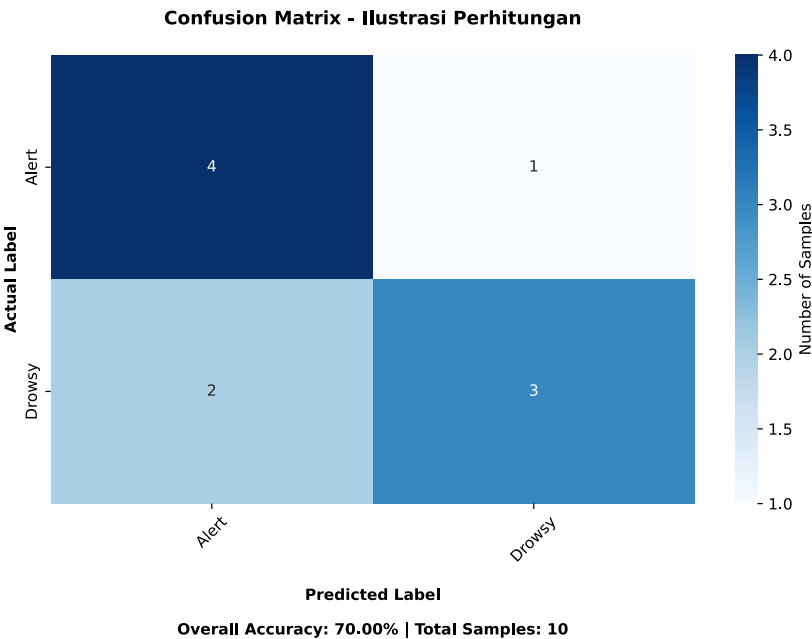
Setelah melalui blok Mamba, *Sequence Output* berdimensi (480×32) diteruskan ke tahap *Global Sequence Aggregation*. Pada tahap ini, data diproses melalui *LayerNorm* dan *AdaptiveAvgPool1D* untuk menghasilkan *Output Vector* tunggal berdimensi (1×32) . Bergantung pada jenis tugas, vektor ini akan diteruskan ke salah satu dari dua *head* yang berbeda. Jika yang sedang dikerjakan adalah tugas klasifikasi, maka vektor akan diteruskan ke *Classification Head* yang terdiri dari lapisan *Linear* ($32 \rightarrow 16$), GELU, *Dropout*, dan *Linear* $16(\rightarrow 2)$ untuk menghasilkan probabilitas kelas secara langsung. Sedangkan jika yang sedang dikerjakan adalah tugas regresi, maka vektor akan diteruskan ke *Regression Head* yang terdiri dari lapisan *Lienar* ($32 \rightarrow 16$), GELU, *Dropout*, dan *Linear* ($16 \rightarrow 1$) untuk menghasilkan skor kontinu.

Khusus untuk pendekatan regresi, nilai keluaran kontinu tersebut kemudian diproses melalui tahap *Thresholding* untuk menghasilkan keputusan akhir berupa *Output (Predicted Class)*. Seluruh alur ini dievaluasi menggunakan skema *subject-wise 5-Fold Cross-Validation* untuk memastikan model mampu melakukan generalisasi terhadap kondisi *Alert* atau *Drowsy* dari subjek yang belum pernah dilihat sebelumnya.

3.5 Ilustrasi Perhitungan

Untuk memberikan pemahaman yang lebih jelas tentang proses perhitungan metrik evaluasi, bagian ini menyajikan ilustrasi perhitungan *end-to-end* menggunakan sampel data simulasi.

Berikut ini adalah contoh *confusion matrix* yang digunakan sebagai contoh ilustrasi perhitungan evaluasi.



Gambar 3.3 Ilustrasi Confusion Matrix

Confusion Matrix yang disajikan pada Gambar 3.3 memvisualisasikan kinerja klasifikasi model untuk dua tingkat kantuk: *Alert* dan *Drowsy*. Dalam matriks ini, label aktual disusun sebagai baris sementara label prediksi ditempatkan sebagai kolom, di mana setiap sel merepresentasikan jumlah sampel untuk kombinasi klasifikasi tertentu. Sebagai ilustrasi, angka 4 pada pertemuan baris dan kolom *Alert* (diagonal utama) menunjukkan bahwa model berhasil memprediksi 4 sampel *Alert* dengan tepat. Distribusi nilai dalam keseluruhan matriks ini menjadi landasan fundamental untuk perhitungan metrik evaluasi kuantitatif, meliputi *accuracy*, *precision*, *recall*, dan *f1-score*.

3.5.1 Precision

Precision untuk mengukur ketepatan prediksi setiap kelas. Berdasarkan rumus 2.7, *precision* dihitung pada masing-masing kelas dengan rincian sebagai

berikut.

$$\text{Precision}_{Alert} = \frac{4}{4+2} = \frac{4}{6} \approx 0.67$$

$$\text{Precision}_{Drowsy} = \frac{3}{3+1} = \frac{3}{4} = 0.75$$

3.5.2 Recall

Recall mengukur kemampuan model mengenali sampel yang benar-benar berasal dari suatu kelas. Berdasarkan rumus 2.8, *recall* dihitung pada masing-masing kelas dengan rincian sebagai berikut:

$$\text{Recall}_{Alert} = \frac{4}{4+1} = \frac{4}{5} = 0.80$$

$$\text{Recall}_{Drowsy} = \frac{3}{3+2} = \frac{3}{5} = 0.60$$

3.5.3 F1-Score

F1-score merupakan rata-rata harmonik antara *precision* dan *recall*. Berdasarkan rumus 2.9, *f1-score* dihitung pada masing-masing kelas dengan rincian sebagai berikut:

$$F1_{Alert} = 2 \times \frac{(0.67)(0.80)}{0.67 + 0.80} = \frac{1.072}{1.47} \approx 0.73$$

$$F1_{Drowsy} = 2 \times \frac{(0.75)(0.60)}{0.75 + 0.60} = \frac{0.90}{1.35} \approx 0.67$$

3.5.4 Accuracy

Accuracy menggambarkan proporsi prediksi benar dibandingkan seluruh sampel. Berdasarkan rumus 2.6, *accuracy* dihitung dengan rincian sebagai berikut:

$$\text{Accuracy} = \frac{4+3}{10} = \frac{7}{10} = 0.70$$

3.6 Rancangan Pengujian

Proses pengujian terhadap model yang telah dilatih dilakukan secara menyeluruh dengan mencakup dua aspek utama, yaitu kinerja klasifikasi dan efisiensi komputasi model. Kinerja klasifikasi dievaluasi berdasarkan *confusion matrix* yang menjadi dasar perhitungan empat metrik evaluasi utama, yaitu *accuracy*, *precision*, *recall*, dan *F1-score*.

Selain itu, untuk menilai efisiensi komputasi dari sisi arsitektural, evaluasi dilakukan menggunakan metrik jumlah parameter (*parameter count*) dan *Giga Floating Point Operations* (GFLOPs). Kedua metrik ini digunakan untuk merepresentasikan kompleksitas model secara perangkat-agnostik, sehingga memungkinkan perbandingan efisiensi komputasi yang lebih adil dan konsisten antar arsitektur.

BAB IV

HASIL DAN PEMBAHASAN

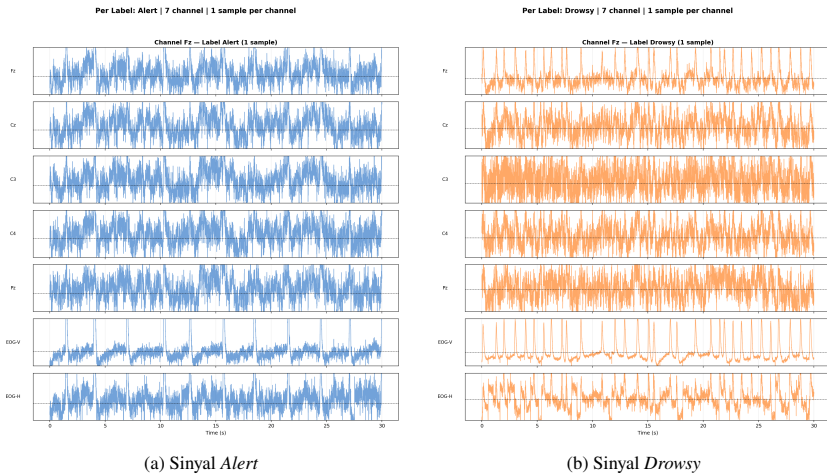
4.1 Hasil Penelitian

Pada bagian ini, akan dijelaskan hasil dari setiap tahapan penelitian yang telah dilakukan, mulai dari penyiapan data hingga proses pelatihan model. Pembahasan di dalamnya mencakup bagaimana data akuisisi dataset sinyal, bagaimana data tersebut dibagi menggunakan skema *5-Fold Cross-Validation*, serta proses pra-pemrosesan data untuk menstandarkan sinyal dari berbagai subjek. Selain itu, sub-bab ini juga menampilkan hasil pelatihan (*training*) model berupa grafik *loss* dari kedua skenario model Mamba (regresi dan klasifikasi). Grafik tersebut digunakan untuk melihat apakah model berhasil belajar dengan baik dan stabil selama proses pelatihan berlangsung.

4.1.1 Akuisisi Dataset

Dataset yang digunakan dalam penelitian ini merupakan dataset publik bertajuk "The ULg Multimodality Drowsiness Database" (DROZY) yang diperoleh melalui platform terbuka untuk penelitian pemantauan tingkat kantuk [5]. Dataset ini mencakup rekaman sinyal fisiologis dari 14 subjek sehat yang menjalani serangkaian sesi eksperimen dalam kondisi kekurangan tidur untuk menginduksi rasa kantuk secara alami. Bahan penelitian yang digunakan terdiri dari 36 set data valid dalam format .edf yang menyertakan label kantuk subjektif serta rekaman *Polysomnography* (PSG). Sinyal fisiologis yang diekstraksi dari dataset ini meliputi 5 kanal EEG (Fz, Cz, Pz, C3, C4) dan 2 kanal EOG (vertikal dan horizontal) dengan frekuensi sampling sebesar 512 Hz. Untuk kebutuhan klasifikasi, label asli dari *Karolinska Sleepiness Scale* (KSS) yang memiliki 9 tingkatan dikategorikan ulang menjadi dua kelas utama, yaitu Alert (KSS 1-5) dan Drowsy (KSS 6-9). Visualisasi perbandingan contoh sinyal mentah

dari ketujuh kanal untuk setiap kategori tingkat kantuk tersebut disajikan pada Gambar 4.1.



Gambar 4.1 Contoh Sinyal dari Kedua Kelas Tingkat Kantuk

4.1.2 Pembagian Dataset

Dataset yang telah disegmentasi dibagi menjadi lima bagian (fold) menggunakan skema Subject-Wise K-Fold Cross Validation dengan nilai $k = 5$. Melalui pendekatan ini, seluruh rekaman yang berasal dari satu subjek ditempatkan secara eksklusif pada satu fold yang sama untuk menjaga kemandirian data uji terhadap karakteristik individu. Total rekaman yang digunakan adalah 36 sesi dari 14 subjek yang menghasilkan sejumlah segmen berdurasi 30 detik. Detail pembagian subjek dan persebaran jumlah sampel untuk setiap kelas pada masing-masing fold disajikan dalam Tabel 4.1 .

Tabel 4.1 Skema Pembagian Subjek pada 5-Fold Cross-Validation

Fold ke-	Data Latih (Subjek)	Data Validasi (Subjek)
1	1, 10, 11, 13, 14, 2, 3, 5, 6, 7, 8	12, 4, 9
2	1, 10, 12, 13, 14, 2, 4, 5, 6, 7, 9	11, 13, 8
3	1, 11, 12, 13, 14, 3, 4, 5, 6, 8, 9	10, 2, 7
4	10, 11, 12, 13, 2, 3, 4, 5, 7, 8, 9	1, 14, 6
5	1, 10, 11, 12, 14, 2, 3, 4, 6, 7, 8, 9	13, 5

4.1.3 Prapemrosesan Data

Pada tahap ini, dataset melalui proses prapemrosesan sebelum dimasukkan ke dalam model untuk tahapan pelatihan. Tahapan ini sangat krusial guna mengubah data sinyal PSG mentah menjadi format sekuensial terstruktur yang siap diolah oleh arsitektur Mamba. Langkah pertama yang dilakukan adalah segmentasi sinyal menggunakan teknik *sliding window*. Sinyal kontinu dari 7 kanal (5 kanal EEG dan 2 kanal EOG) dipotong menjadi segmen-segmen dengan *window size* selama 30 detik. Pemotongan ini diatur menggunakan parameter pergeseran (*stride*) sebesar 10 detik, yang secara otomatis menghasilkan area *overlap* antar-segmen yang berdekatan sebesar 20 detik. Berdasarkan frekuensi *sampling* bawaan sebesar 512 Hz, setiap segmen 30 detik tersebut akan diekstrak menjadi matriks atau tensor berukuran $[7 \times 15.360]$ titik data, yang kemudian diperlakukan sebagai satu sampel data individual. Pendekatan *overlap* ini sangat efektif untuk melipatgandakan jumlah sampel data sekaligus mempertahankan kontinuitas konteks temporal perubahan fase kantuk dari waktu ke waktu. Detail implementasi algoritma *sliding window* pada proses segmentasi dataset ini direpresentasikan pada Kode 4.1.

Kode 4.1 Kode Segmentasi Data

1

2

```
# config.py
WINDOW_SEC = 30
```

```

3 STRIDE_SEC = 10
4
5 # train.py
6 window_sec=Config.WINDOW_SEC
7 stride_sec=Config.STRIDE_SEC
8
9 try:
10     raw_info = mne.io.read_raw_edf(file_path, preload=False,
11     verbose='error')
12     duration_sec = raw_info.times[-1]
13
14     max_start = duration_sec - self.window_sec
15
16     if max_start <= 0:
17         print(f" [SKIP] {filename}: terlalu pendek
18         ({duration_sec:.1f}s)")
19         continue
20
21     subject_id = filename.split('-')[0]
22     for start in np.arange(0, max_start, self.stride_sec):
23         self.samples.append({
24             'file_path': file_path,
25             'start_sec': float(start),
26             'label': label,
27             'class_idx': kss_to_class(label),
28             'subject_id': subject_id
29         })
30         total_windows += 1
31 except Exception as e:
32     print(f" [ERROR] {filename}: {e}")

```

Langkah kedua adalah penerapan Normalisasi Z-Score atau standardisasi untuk menyeragamkan skala setiap fitur sinyal. Dalam penelitian ini, normalisasi dilakukan secara spesifik per subjek untuk mengeliminasi perbedaan karakteristik neurofisiologis antar individu yang dapat memengaruhi objektivitas model. Dengan menyamakan distribusi data melalui nilai rata-rata (mean) dan standar deviasi, metode ini berfungsi untuk memitigasi masalah pergeseran domain antara data pelatihan dan data pengujian. Untuk menjaga validitas evaluasi dan mencegah kebocoran data, parameter statistik (μ dan σ) dihitung hanya berdasarkan data latih pada setiap fold dalam skema subject-wise K-Fold Cross Validation, yang kemudian diaplikasikan secara konsisten pada data uji di fold yang bersesuaian. Implementasi kode prapemrosesan ini disajikan pada Kode 4.2.

Kode 4.2 Kode Normalisasi

```

1 norm_mode = getattr(Config, 'NORMALIZATION', 'window')
2 if norm_mode == 'subject' and hasattr(self, 'subject_stats'):
3     subject_id = info.get('subject_id',
4     os.path.basename(file_path).split('-')[0])
5     subject_stats = self.subject_stats.get(subject_id)

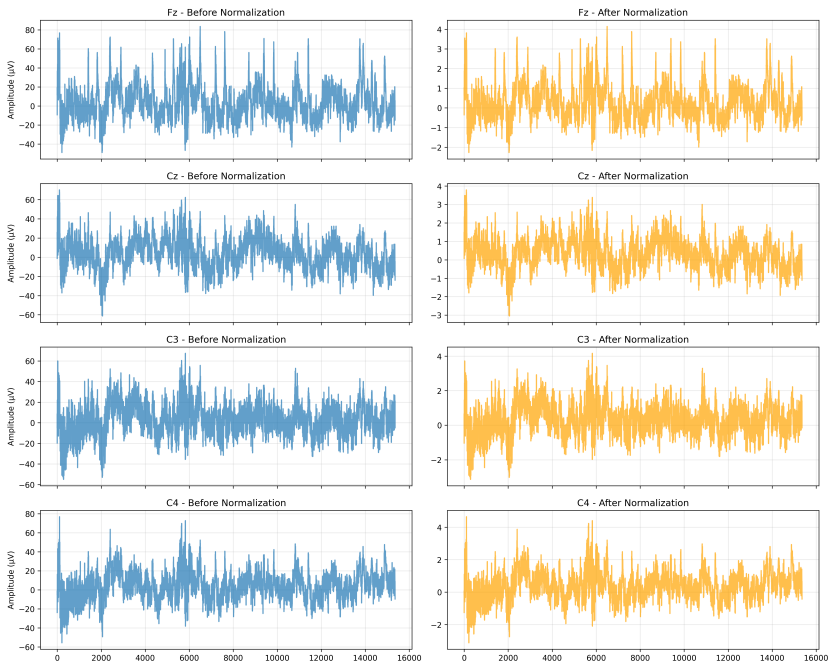
```

```

5         if subject_stats is not None:
6             mean, std = subject_stats
7             signal = (signal - mean) / (std + 1e-6)
8         else:
9             mean = np.mean(signal, axis=1, keepdims=True)
10            std = np.std(signal, axis=1, keepdims=True)
11            signal = (signal - mean) / (std + 1e-6)

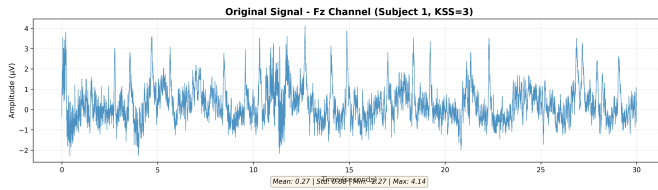
```

Gambar 4.2 menyajikan representasi visual dari hasil penerapan normalisasi sinyal per subjek. Grafik pada sisi kiri menampilkan kondisi sinyal mentah sebelum dinormalisasi, yang masih memiliki rentang skala amplitudo yang sangat lebar. Sebaliknya, grafik pada sisi kanan menunjukkan sinyal setelah dinormalisasi, di mana rentang nilai amplitudonya telah menyusut secara signifikan dan berskala jauh lebih seragam.

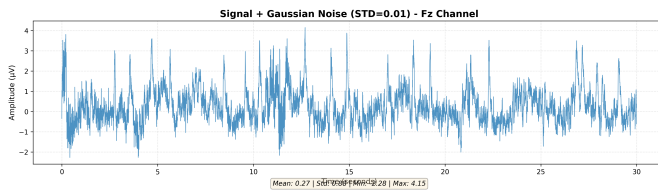


Gambar 4.2 Perbandingan Sinyal Sebelum & Sesudah Normalisasi

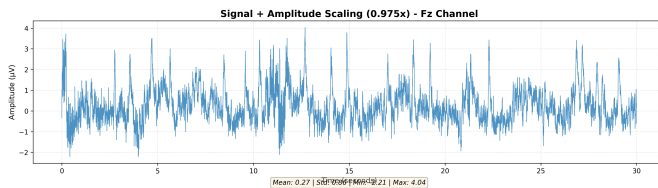
4.1.4 Augmentasi Data



(a) Sinyal Original



(b) Sinyal dengan Gaussian Noise



(c) Sinyal dengan Amplitude Scaling

Gambar 4.3 Sinyal Augmentasi

Sesuai dengan skenario yang telah ditetapkan, proses augmentasi data diterapkan secara dinamis pada setiap iterasi pelatihan untuk memperluas keragaman distribusi data tanpa membebani kapasitas memori. Untuk memverifikasi dampak transformasi terhadap karakteristik fundamental sinyal biologis, dilakukan inspeksi visual dan statistik, seperti yang diilustrasikan pada Gambar 4.3.

Secara kasat mata, sinyal asli, sinyal dengan penambahan derau (*Gaussian Noise*), dan sinyal yang diskalakan (*Amplitude Scaling*) tampak identik tanpa adanya perubahan bentuk gelombang yang drastis. Hal ini membuktikan bahwa strategi *mild augmentation* yang diterapkan berhasil menjaga integritas fitur

temporal dari sinyal EEG. Namun, modifikasi tersebut dapat diamati secara jelas melalui parameter statistik sinyal.

Pada penambahan *Gaussian Noise* dengan standar deviasi 0.01, terjadi pergeseran minor pada nilai ekstrem amplitudo; nilai minimum bergeser dari -2.27 menjadi -2.28, dan nilai maksimum dari 4.14 menjadi 4.15. Sementara itu, pada penerapan *Amplitude Scaling* dengan faktor pengali 0.975x, rentang amplitudo mengalami penyusutan proporsional, yang ditandai dengan perubahan nilai minimum menjadi -2.21 dan maksimum menjadi 4.04. Meskipun perubahannya sangat kecil secara visual, variasi numerik ini sudah cukup efektif untuk memaksa model Mamba agar tidak menghafal nilai absolut (*overfitting*), melainkan berfokus pada ekstraksi pola dari gelombang kantuk tersebut.

Seluruh proses transformasi probabilitas ini dieksekusi secara terprogram pada tahapan pembacaan data. Detail implementasi algoritma augmentasi tersebut diwujudkan melalui skrip Python yang ditunjukkan pada Kode 4.3 berikut.

Kode 4.3 Kode Augmentasi

```

1  # config.py
2  USE_AUGMENTATION = True
3  AUG_GAUSSIAN_NOISE_STD = 0.01
4  AUG_AMPLITUDE_SCALE_RANGE = (0.9, 1.1)
5
6  # datareader.py
7  class EEGAugmentation:
8      def __init__(
9          self,
10         gaussian_std=Config.AUG_GAUSSIAN_NOISE_STD,
11         amplitude_range=Config.AUG_AMPLITUDE_SCALE_RANGE,
12         time_shift_max=Config.AUG_TIME_SHIFT_MAX,
13         prob=0.5
14     ):
15         self.gaussian_std = gaussian_std
16         self.amplitude_range = amplitude_range
17         self.time_shift_max = time_shift_max
18         self.prob = prob
19
20     def add_gaussian_noise(self, signal):
21         if np.random.rand() < self.prob:
22             noise = torch.randn_like(signal) * self.gaussian_std
23             signal = signal + noise
24         return signal
25
26     def amplitude_scaling(self, signal):
27         if np.random.rand() < self.prob:
28             scale = np.random.uniform(*self.amplitude_range)
29             signal = signal * scale
30         return signal

```

```

31
32 # train.py
33 train_dataset = EEGDataset(
34     data_dir=Config.DATA_DIR,
35     csv_path=os.path.join('label/labels.csv'),
36     fold=fold,
37     split='train',
38     n_splits=Config.N_SPLITS,
39     window_sec=Config.WINDOW_SEC,
40     stride_sec=Config.STRIDE_SEC,
41     use_augmentation=Config.USE_AUGMENTATION
42 )
43
44 val_dataset = EEGDataset(
45     data_dir=Config.DATA_DIR,
46     csv_path=os.path.join('label/labels.csv'),
47     fold=fold,
48     split='val',
49     n_splits=Config.N_SPLITS,
50     window_sec=Config.WINDOW_SEC,
51     stride_sec=Config.WINDOW_SEC,
52     use_augmentation=False
53 )
54
55 train_loader = DataLoader(
56     train_dataset
57 )
58
59 val_loader = DataLoader(
60     val_dataset
61 )

```

4.1.5 Hasil Pelatihan Model

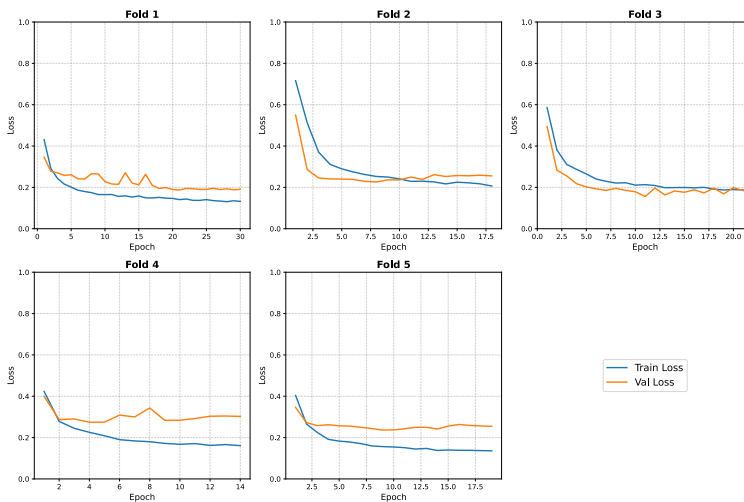
Setelah seluruh tahapan prapemrosesan dan augmentasi data diselesaikan, model hibrida regresi dan klasifikasi berbasis arsitektur Mamba memasuki tahap pelatihan utama. Proses pelatihan dieksekusi dengan menggunakan *hyperparameter* yang telah dioptimalkan secara empiris guna menjaga keseimbangan antara *convergence rate* dan pencegahan *overfitting*. Rincian *hyperparameter* yang digunakan pada eksperimen ini dirangkum pada Tabel 4.2.

Tabel 4.2 Konfigurasi *Hyperparameter* Pelatihan Model Mamba

Parameter	Nilai
<i>Epoch</i> Maksimal	50
<i>Batch Size</i>	32
<i>Optimizer</i>	AdamW
<i>Learning Rate</i>	1×10^{-4}
<i>Weight Decay</i>	1×10^{-2}
<i>Early Stopping Patience</i>	10 <i>epoch</i>
<i>LR Scheduler</i>	ReduceLROnPlateau

Proses optimasi model menggunakan *optimizer* AdamW dengan *Weight Decay* sebesar 1×10^{-2} sebagai bentuk regulisasi untuk mengurangi ketergantungan model pada *noise* sinyal. Strategi pelatihan dilengkapi dengan *Learning Rate Scheduler* menggunakan metode *ReduceLROnPlateau*, yang secara otomatis menurunkan laju pembelajaran saat nilai *loss* validasi mengalami stagnasi guna mencapai konvergensi yang lebih halus. Selain itu, diterapkan mekanisme *Early Stopping* dengan *patience* sebanyak 10 *epoch* untuk menghentikan pelatihan saat tidak terjadi perbaikan performa, sehingga mencegah *overfitting* dan memastikan model tersimpan pada kondisi optimal.

Sebagai representasi visual dari dinamika pembelajaran model, Gambar 4.4 menampilkan kurva penurunan kerugian (*loss curve*) untuk target regresi yang dievaluasi menggunakan *Mean Absolute Error* (MAE). Grafik ini diambil dari *Fold* ke-1 hingga ke-5.

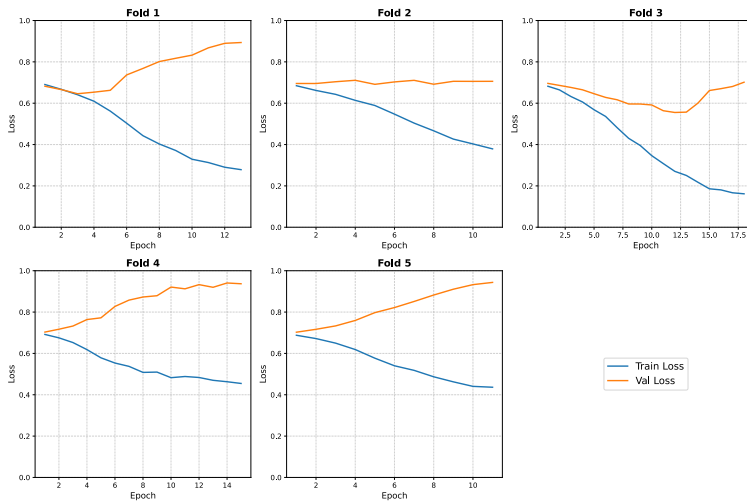


Gambar 4.4 Grafik Loss Pendekatan Regresi

Berdasarkan visualisasi keseluruhan grafik *loss* dari kelima *fold* pengujian tersebut, dapat diamati bahwa nilai *training loss* secara konsisten mengalami penurunan yang signifikan dan mulus hingga mencapai titik stabil di kisaran

0.15 hingga 0.20 pada akhir *epoch*. Sementara itu, kurva *validation loss* secara umum juga mengikuti tren penurunan yang searah dan mampu berkonvergensi dengan baik di kisaran angka 0.20 hingga 0.30. Meskipun terdapat fluktuasi minor pada kurva validasi di beberapa *fold*, hal tersebut merupakan karakteristik wajar akibat tingginya kompleksitas dan variabilitas data sinyal fisiologis antar-individu. Jarak yang relatif sempit dan stabil antara kurva latih dan kurva validasi di seluruh *fold* ini menjadi bukti empiris bahwa cabang regresi pada arsitektur Mamba berhasil mengekstrak fitur temporal secara efektif tanpa mengalami *overfitting*. Hal ini sekaligus mengonfirmasi keberhasilan fungsi regularisasi *Weight Decay* yang diterapkan dalam menjaga stabilitas generalisasi model di berbagai variasi data subjek.

Berbeda dengan target regresi, cabang klasifikasi pada model menunjukkan dinamika pelatihan yang lebih menantang. Gambar 4.5 menampilkan kurva kerugian untuk target klasifikasi yang dievaluasi menggunakan *Weighted Cross-Entropy Loss*.



Gambar 4.5 Grafik Loss Pendekatan Klasifikasi

Berdasarkan visualisasi keseluruhan grafik *loss* dari kelima *fold* pengujian

tersebut, terlihat pola umum di mana *training loss* selalu berhasil menurun secara konsisten. Namun, kurva *validation loss* mayoritas menunjukkan tren peningkatan pada titik *epoch* yang bervariasi, seperti setelah *epoch* ke-5 pada Fold 1, setelah *epoch* ke-12 pada Fold 3, hingga tren yang langsung meningkat sejak awal pelatihan pada Fold 4 dan 5. Fenomena kenaikan yang terjadi secara sistematis di hampir seluruh *fold* ini sangat dipengaruhi oleh tingginya variabilitas karakteristik sinyal antar-subjek, serta perbedaan karakteristik matematis antara kedua fungsi kerugian yang digunakan. Pada cabang regresi, MAE menghitung selisih kesalahan secara absolut dan *linier*, sehingga penalti yang diberikan terhadap prediksi yang meleset tetap proporsional dan menjaga kurva tetap stabil.

Sebaliknya, pada cabang klasifikasi biner, model dituntut untuk membuat keputusan kategori yang kaku dan tegas. Fenomena lonjakan *validation loss* ini sangat dipengaruhi oleh sifat fungsi kerugian yang memberikan hukuman besar pada batas transisi antar-kelas. Sebagai perbandingan, pada skenario regresi, jika skor KSS aktual subjek adalah 5 namun model memprediksi 6, kesalahan tersebut hanya dihitung sebagai selisih kecil yang proporsional sehingga kurva tetap stabil. Namun pada skenario klasifikasi, jika skor aktual adalah 5 (kategori *Alert*) dan fitur model sedikit bergeser menyerupai skor 6 (kategori *Drowsy*), model akan langsung dianggap melakukan kesalahan fatal karena tebakannya menyeberang ke kelas yang berbeda. Mengingat transisi fisiologis sinyal kantuk pada dataset DROZY terjadi secara bertahap dan sangat halus, model kerap kali terjebak pada batas tipis ini saat memproses data validasi dari subjek baru. Pemberian penalti yang besar akibat "salah kamar" pada batas transisi inilah yang menyebabkan akumulasi *validation loss* melonjak tajam secara visual, meskipun secara fisiologis tebakan model sebenarnya hanya meleset tipis dari ambang batas.

4.2 Hasil Evaluasi Kinerja Model

Evaluasi kinerja model dieksekusi menggunakan skema *5-Fold Cross-Validation* untuk memastikan bahwa performa yang dihasilkan bersifat objektif dan tidak bias terhadap porsi pembagian data tertentu. Mengingat arsitektur Mamba yang diusulkan memiliki dua cabang keluaran (regresi dan klasifikasi), evaluasi dilakukan secara terpisah untuk membandingkan efektivitas kedua pendekatan tersebut dalam mendeteksi tingkat kantuk.

4.2.1 Kinerja Pendekatan Regresi

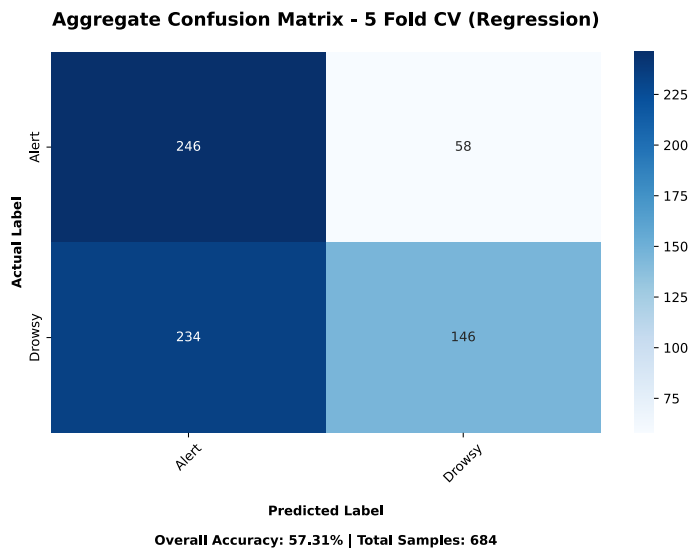
Pada cabang regresi, model bertugas memprediksi nilai *Karolinska Sleepiness Scale* (KSS) dalam rentang kontinu 1 hingga 9. Untuk dapat membandingkan kinerjanya dengan tugas klasifikasi biner, nilai KSS hasil prediksi regresi dikonversi menggunakan nilai ambang batas (*threshold*) sebesar 5.5, sebagaimana telah dijustifikasi pada bab metodologi. Apabila nilai KSS prediksi ≤ 5.5 , maka sampel dikategorikan sebagai *Alert* (rentang asli 1-5), sedangkan jika nilai > 5.5 dikategorikan sebagai *Drowsy* (rentang asli 6-9). Rangkuman hasil evaluasi regresi untuk kelima *fold* disajikan pada Tabel 4.3.

Tabel 4.3 Hasil Evaluasi 5-Fold Cross-Validation pada Target Regresi

Fold	MAE (Skala 1-9)	Akurasi Threshold (%)	F1-Score
1	1.6926	64.91	0.6429
2	1.9560	54.39	0.3158
3	1.5990	63.91	0.5385
4	2.3994	53.80	0.2883
5	2.1225	50.53	0.5524
Rata-rata	1.9539 $\pm$ 0.2902	57.51 $\pm$ 6.48	0.4676 $\pm$ 0.1566

Berdasarkan Tabel 4.3, model mencapai rata-rata MAE sebesar 1.9539. Perlu dicatat bahwa meskipun model dilatih menggunakan target KSS yang dinormalisasi ke rentang 0-1, perhitungan metrik evaluasi MAE akhir dilakukan setelah mengembalikan nilai prediksi dan target ke skala KSS asli (1-9) melalui proses *inverse transform* (dikalikan dengan nilai maksimum 9.0). Hal ini

dilakukan agar nilai *error* dapat diinterpretasikan secara langsung. Angka MAE ini menunjukkan bahwa prediksi model secara umum meleset sekitar 2 poin dari skor KSS aktual subjek. Meskipun terdapat variansi performa antar-*fold*, stabilitas MAE yang berada di bawah angka 2.5 membuktikan bahwa model mampu mengikuti tren perubahan sinyal EEG dan EOG secara bertahap. Namun, ketika hasil regresi dikonversi menjadi klasifikasi biner, rata-rata akurasi yang dihasilkan adalah 57,51% ($\pm 6,48\%$) dengan F1-Score sebesar 0,4676 ($\pm 0,1566$). Tingginya angka standar deviasi ini menjadi indikasi awal adanya fluktuasi performa yang dipengaruhi oleh karakteristik subjek pada masing-masing *fold*. Di samping variansi tersebut, rendahnya nilai rata-rata F1-Score juga mengindikasikan adanya ketidakseimbangan model dalam mendeteksi kelas tertentu, yang diperjelas melalui analisis *Confusion Matrix* pada Gambar 4.6.



Gambar 4.6 Confusion Matrix Agregasi Pendekatan Regresi

Berdasarkan hasil agregasi *Confusion Matrix* dari kelima *fold* pengujian dengan total 684 sampel, pendekatan regresi menghasilkan akurasi sebesar

57,31%. Dari Gambar 4.6 di atas terlihat bahwa model memiliki kecenderungan untuk lebih sering memprediksi kelas sadar (*Alert*). Model berhasil mengenali kondisi *Alert* dengan cukup baik, yaitu 246 prediksi benar dari 304 sampel asli. Namun, kelemahan utama model ini terlihat jelas pada kemampuannya mendeteksi kondisi kantuk (*Drowsy*). Dari 380 sampel yang aslinya *Drowsy*, model hanya mampu memprediksi dengan benar sebanyak 146 sampel, sedangkan 234 sampel lainnya salah diklasifikasikan sebagai *Alert*. Tingginya angka kesalahan ini (*False Negative*) menunjukkan bahwa model regresi terlalu berhati-hati dan lebih memilih untuk "mencari aman" dengan memprediksi kondisi *Alert*. Hal ini membuktikan bahwa penggunaan fungsi kerugian MAE pada model regresi kurang peka dalam menangkap tanda-tanda awal dari rasa kantuk. Meskipun sangat akurat dalam mencegah *False Alarm*, pendekatan ini memiliki risiko tinggi karena gagal mendeteksi subjek yang sebenarnya sudah mulai kehilangan kesadaran.

4.2.2 Kinerja Pendekatan Klasifikasi

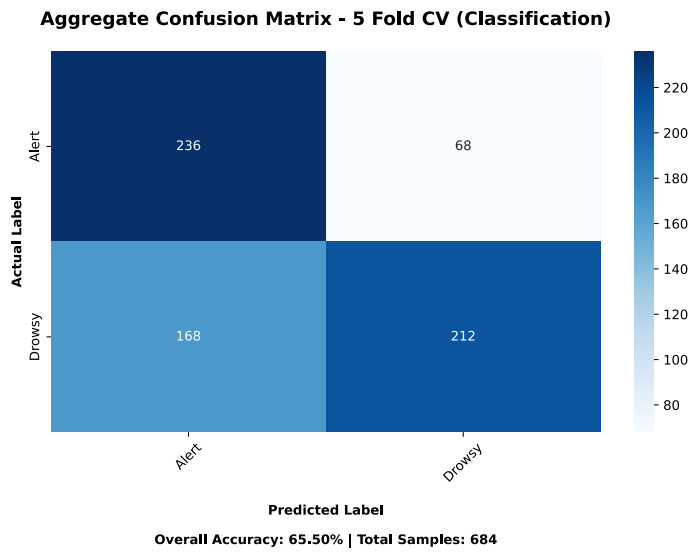
Berbeda dengan cabang regresi yang memprediksi nilai kontinu secara bertahap, cabang klasifikasi pada arsitektur Mamba dilatih secara khusus menggunakan *Weighted Cross-Entropy Loss* untuk langsung memisahkan sinyal ke dalam dua kelas biner (*Alert* dan *Drowsy*). Hasil evaluasi performa klasifikasi dari kelima *fold* dirangkum secara mendetail pada Tabel 4.4.

Tabel 4.4 Hasil Evaluasi 5-Fold Cross-Validation pada Target Klasifikasi

Fold	Akurasi (%)	F1-Score
1	80.70	0.7512
2	61.99	0.5485
3	69.17	0.6860
4	60.23	0.5698
5	57.89	0.3667
Rata-rata	66.00 ± 8.26	0.5844 ± 0.1319

Berdasarkan Tabel 4.4, pendekatan klasifikasi murni menghasilkan rata-

rata akurasi sebesar 66,00% ($\pm 8,26\%$). Jika dibandingkan dengan rata-rata akurasi hasil *thresholding* regresi (57,51%), angka ini secara keseluruhan terlihat lebih tinggi. Namun, nilai rata-rata F1-Score yang hanya sebesar 0,5844 ($\pm 0,1319$) menunjukkan adanya masalah keseimbangan dalam prediksi. Besarnya rentang standar deviasi pada metrik ini mengonfirmasi bahwa performa model sangat fluktuatif. Model mampu memprediksi dengan tingkat keberhasilan yang tinggi di beberapa *fold*, namun performanya merosot tajam di *fold* lainnya akibat perbedaan karakteristik subjek yang diuji. Untuk memahami detail distribusi kesalahan dan bagaimana performa model secara keseluruhan, kita perlu melihat agregasi tebakan model pada *Confusion Matrix* di Gambar 4.7.



Gambar 4.7 Confusion Matrix Agregasi Pendekatan Klasifikasi

Hasil agregasi *Confusion Matrix* pada pendekatan klasifikasi biner menunjukkan kinerja yang lebih seimbang dengan tingkat akurasi yang lebih tinggi, yaitu 65,50%. Berdasarkan Gambar 4.7, penggunaan fungsi kerugian *Weighted Cross-Entropy* terbukti mampu membuat model lebih fokus untuk

mengenal kelas *Drowsy*. Jumlah prediksi benar untuk kondisi kantuk meningkat secara nyata menjadi 212 sampel, jauh lebih baik dibandingkan dengan pendekatan regresi. Sebagai risikonya, model ini menjadi sedikit lebih sensitif sehingga memunculkan lebih banyak alarm palsu (*False Positive*), di mana terdapat 68 sampel *Alert* yang salah diprediksi sebagai *Drowsy*. Walaupun masih ada 168 sampel kantuk yang belum berhasil terdeteksi, secara keseluruhan pendekatan klasifikasi ini memberikan distribusi prediksi yang lebih merata. Pendekatan klasifikasi dinilai lebih sesuai untuk sistem peringatan keselamatan karena lebih sigap dan tanggap dalam menangkap indikasi pengemudi yang mulai mengantuk.

Dapat disimpulkan bahwa kedua pendekatan ini memiliki karakter yang bertolak belakang. Model regresi cenderung sangat berhati-hati dan memprioritaskan untuk meminimalkan salah prediksi subjek sadar sebagai mengantuk, meskipun risikonya adalah banyak subjek yang sebenarnya mengantuk justru terlewat. Sementara itu, model klasifikasi bertindak lebih responsif dengan fokus untuk "menangkap sebanyak mungkin" indikasi kantuk, walaupun hal ini memicu sedikit peningkatan pada jumlah alarm palsu. Perbedaan performa yang kontras ini membuktikan bahwa mendeteksi transisi kantuk dari sinyal EEG dan EOG merupakan tugas yang sangat menantang. Batas fisiologis antara kondisi sadar dan mulai mengantuk sangatlah tipis, sehingga secara tidak langsung model dipaksa untuk memilih sebuah kompromi: menjadi terlalu berhati-hati seperti pada pendekatan regresi, atau lebih agresif seperti pada klasifikasi.

4.2.3 Perbandingan Metrik Per-Kelas

Untuk memperoleh pemahaman yang lebih mendalam mengenai karakteristik prediksi masing-masing pendekatan, evaluasi dilakukan pada tingkat per-kelas menggunakan data agregasi dari kelima *fold* pengujian. Tabel 4.5 menyajikan nilai *precision*, *recall*, dan *F1-score* untuk setiap kelas

pada kedua pendekatan secara terpisah.

Tabel 4.5 Perbandingan Metrik Per-Kelas Pendekatan Regresi dan Klasifikasi

Pendekatan	Kelas	Precision	Recall	F1-Score
Regresi	<i>Alert</i>	0,5125	0,8092	0,6277
	<i>Drowsy</i>	0,7157	0,3842	0,4998
	Macro Avg	0,6141	0,5967	0,5638
Klasifikasi	<i>Alert</i>	0,5842	0,7763	0,6666
	<i>Drowsy</i>	0,7571	0,5579	0,6425
	Macro Avg	0,6707	0,6671	0,6546

Berdasarkan Tabel 4.5, terdapat pola yang sangat kontras antara kedua pendekatan dalam mendeteksi masing-masing kelas. Pada pendekatan regresi, model menunjukkan nilai *recall* yang tinggi untuk kelas *Alert* (0,8092), namun *recall* kelas *Drowsy* hanya mencapai 0,3842. Artinya, dari seluruh sampel yang sebenarnya berstatus mengantuk, model regresi hanya mampu mengidentifikasi dengan benar sekitar 38,42% di antaranya. Kondisi ini secara langsung mengkonfirmasi temuan pada analisis *confusion matrix* sebelumnya, bahwa model regresi berperilaku layaknya *conservative classifier* yang cenderung memilih prediksi *Alert* sebagai opsi yang lebih aman.

Sebaliknya, pendekatan klasifikasi menunjukkan distribusi metrik yang jauh lebih seimbang antar kelas. Nilai *recall* kelas *Drowsy* meningkat secara signifikan menjadi 0,5579 dibandingkan pendekatan regresi, yang berarti model klasifikasi mampu mendeteksi lebih dari separuh kondisi kantuk yang sesungguhnya. Sebagai konsekuensinya, nilai *precision* kelas *Alert* mengalami penurunan dari 0,5125 menjadi 0,5842 yang mengindikasikan adanya peningkatan jumlah *false alarm* dibandingkan pendekatan regresi.

Dari perspektif *macro average*, pendekatan klasifikasi mengungguli regresi pada seluruh metrik agregat: *precision* (0,6707 vs. 0,6141), *recall* (0,6671 vs.

0,5967), dan *F1-score* (0,6546 vs. 0,5638). Perlu dicatat bahwa nilai *macro F1-score* agregat ini dihitung dari data gabungan kelima *fold* dan berbeda dari rata-rata *F1-score* per-*fold* yang dilaporkan pada Tabel 4.3 dan 4.4. Perbedaan ini wajar secara metodologis karena jumlah sampel per *fold* tidak seragam, sehingga agregasi langsung dan rata-rata aritmetika per-*fold* akan menghasilkan angka yang sedikit berbeda.

4.3 Analisis Efisiensi Komputasi

Keunggulan utama dari penggunaan arsitektur Mamba dalam penelitian ini adalah efisiensi komputasinya yang sangat tinggi. Evaluasi efisiensi dilakukan dengan mengukur parameter struktural model dan beban operasional saat melakukan inferensi. Ringkasan metrik efisiensi model disajikan pada Tabel 4.6.

Tabel 4.6 Metrik Efisiensi Komputasi Arsitektur Mamba

Parameter Efisiensi	Nilai
Total Parameter	62.930 unit
Ukuran File Model (.pth)	820 KB
Operasi Komputasi (GFlops)	0,01

Berdasarkan Tabel 4.6, model Mamba yang dikembangkan memiliki total parameter sebesar 62.930 unit. Jumlah parameter ini dikategorikan sebagai arsitektur yang sangat ringan (*lightweight*), mengingat model klasifikasi standar umumnya memiliki jutaan parameter. Sehingga model sangat kompetitif untuk dijalankan pada perangkat dengan sumber daya terbatas (*Edge Device*).

Dari sisi beban operasional, model ini mencatatkan nilai komputasi sebesar 0,01 GFlops. Nilai yang sangat rendah ini menunjukkan bahwa model dapat memproses sinyal EEG/EOG dengan sangat cepat tanpa membebani unit pemroses grafis (GPU). Dengan demikian, arsitektur Mamba terbukti memenuhi syarat efisiensi untuk aplikasi pemantauan kondisi pengemudi secara berkelanjutan dengan konsumsi daya yang minimal.

4.4 Perbandingan dengan *Baseline*

Untuk mengevaluasi performa model secara menyeluruh, hasil pengujian Mamba pada penelitian ini dibandingkan dengan model pembanding *baseline* dari Cao et al. (2025) [13]. Model tersebut dipilih sebagai pembanding utama karena sama-sama menggunakan dataset DROZY [5] untuk kasus deteksi kantuk. Walaupun menggunakan dataset yang sama, perlu dicatat bahwa perbandingan ini tidak sepenuhnya *apple-to-apple*. Hal ini dikarenakan model pembanding menggunakan pendekatan *multimodal*, sedangkan penelitian ini murni menggunakan pendekatan *single-modal* dengan arsitektur Mamba. Selain perbedaan modalitas, informasi mengenai jumlah parameter dan beban komputasi tidak disebutkan secara eksplisit di dalam jurnal pembanding tersebut. Oleh karena itu, estimasi nilai komputasi pada model *baseline* didapatkan dengan cara mereplikasi ulang arsitekturnya berdasarkan deskripsi struktural yang dilaporkan penulisnya. Meskipun hasil replikasi ini mungkin tidak seratus persen identik dengan kode aslinya, kerangka makro yang dibangun sudah sangat mendekati dan dinilai cukup valid untuk digunakan sebagai acuan komparasi. Pada akhirnya, perbandingan ini tetap penting dilakukan untuk melihat seberapa jauh model Mamba bisa lebih efisien secara komputasi dan seberapa layak model ini diterapkan untuk sistem deteksi kantuk secara *real-time* yang membutuhkan proses ringan.

Tabel 4.7 Perbandingan Kinerja dan Efisiensi Komputasi Model

Metrik Evaluasi	Model Pembanding (Cao et al.)	Model Usulan (Klasifikasi)	Model Usulan (Regresi)
Modalitas Input	Multimodal	<i>Single-modal</i>	<i>Single-modal</i>
Arsitektur Utama	ResNet18 + LSTM	Mamba (4 Layer)	Mamba (4 Layer)
Akurasi	98,41%	66,00%	57,51%
F1-Score	0,9838	0,5844	0,4676
Total Parameter	11,57 juta	62.930 unit	62.930 unit
Beban Komputasi	0,27 GFlops	0,01 GFlops	0,01 GFlops

Berdasarkan tabel 4.7, terlihat adanya *trade-off* yang signifikan antara akurasi dan efisiensi komputasi. Model *multimodal* pembandingan terbukti mampu menghasilkan akurasi klasifikasi yang sangat tinggi (98,41%), namun pencapaian tersebut menuntut beban komputasi yang berat, yakni sekitar 0,27 GFLOPs dengan ukuran model mencapai 11,57 juta parameter. Hal ini disebabkan oleh penggunaan lapisan konvolusi dalam (ResNet18) untuk memproses matriks gambar 2D. Sebaliknya, model Mamba yang diusulkan dalam penelitian ini menawarkan efisiensi komputasi yang ekstrem. Dengan jumlah parameter yang sangat kecil (62.930 unit) dan beban komputasi yang jauh lebih ringan (0,01 GFLOPs), model ini memberikan alternatif pendekatan *lightweight* dalam memproses sinyal EEG/EOG. Efisiensi ini menjadikan model Mamba memiliki potensi sumber daya terbatas (*edge devices*) di dalam kendaraan, yang mana kecepatan inferensi dan konsumsi daya yang rendah menjadi prioritas utama.

Selain perbedaan pada modalitas, perbedaan hasil metrik ini juga dipengaruhi oleh konteks *setup* eksperimen. Model pembandingan melakukan ekstraksi fitur gambar wajah secara bersamaan dengan pemrosesan sinyal fisiologis, yang secara inheren memberikan lebih banyak petunjuk visual seperti kondisi mata yang terpejam atau posisi kepala secara langsung kepada model. Di sisi lain, model pada penelitian ini lebih fokus untuk melihat kemampuan asli arsitektur Mamba dalam memproses data sinyal fisiologis secara berurutan. Penggunaan teknik pemotongan data dan CNN (*Temporal CNN Feature Extractor*) di awal proses terbukti sangat membantu meringankan beban kerja model. Langkah ini berhasil memangkas proses komputasi tanpa membuang informasi penting dari pola gelombang otak (EEG) dan gerakan mata (EOG). Pada akhirnya, walaupun tingkat akurasi yang didapat untuk skema klasifikasi dan regresi belum melampaui model *multimodal*, studi ini berhasil membuktikan bahwa Mamba bisa berjalan dengan sumber daya komputasi yang jauh lebih hemat dan efisien.

4.5 Analisis Hasil Penelitian

4.5.1 Analisis Karakteristik Regresi dan Klasifikasi

Berdasarkan perbandingan hasil evaluasi, terlihat bahwa kedua pendekatan dalam arsitektur Mamba ini menampilkan karakteristik yang sangat bertolak belakang akibat perbedaan fungsi kerugian yang digunakan. Cabang regresi yang dilatih dengan fungsi kerugian MAE terbukti memiliki kecenderungan lebih berhati-hati. Tingginya tingkat deteksi pada kondisi *Alert* bukanlah indikator keunggulan mutlak, melainkan dampak dari kehati-hatian model dalam memberikan prediksi skor tinggi, sehingga model berperilaku layaknya *majority class classifier*. Sifat ini membuat model regresi sangat minim menghasilkan *False Alarm*, namun kurang optimal untuk dijadikan sistem peringatan dini karena rentan melewatkan fase awal kantuk.

Di sisi lain, cabang klasifikasi yang dilatih menggunakan *Weighted Cross-Entropy* menunjukkan sifat yang lebih tanggap dan agresif dalam mendeteksi kondisi *Drowsy*. Fungsi pembobotan kelas berhasil memaksa model untuk lebih memprioritaskan kelas minoritas atau kondisi bahaya, meskipun hal ini membawa risiko peningkatan kesalahan prediksi (*False Alarm*) pada subjek yang sebenarnya masih sadar.

Selain dari metrik evaluasi akhir (Akurasi dan F1-Score), kontradiksi karakteristik kedua pendekatan ini juga sangat jelas terlihat pada dinamika kurva kerugian (*loss curve*) selama proses pelatihan. Secara angka performa biner, pendekatan klasifikasi memang lebih unggul, namun grafik *validation loss*-nya (Gambar 4.5) justru menunjukkan tren fluktuatif dan akumulasi *error* yang meningkat. Sebagaimana dijelaskan sebelumnya, lonjakan ini bukanlah indikasi bahwa model gagal belajar, melainkan konsekuensi dari sifat pembagian kelas yang kaku. Model kerap menerima penalti kesalahan yang maksimal ketika prediksi fitur sinyalnya hanya meleset tipis melewati ambang batas kategori. Tingginya lonjakan *loss* ini mengonfirmasi betapa sulitnya memisahkan batas

kondisi sadar dan mengantuk secara tegas pada data EEG dan EOG akibat transisi yang sangat halus.

Sebaliknya, meskipun menghasilkan metrik klasifikasi biner yang lebih rendah setelah melalui proses konversi, pendekatan regresi menampilkan dinamika pembelajaran yang jauh lebih sehat dan stabil secara grafik yang dapat dilihat pada Gambar 4.4. Karena fungsi kerugian MAE hanya menghitung selisih jarak secara murni, prediksi yang meleset tipis akan tetap dievaluasi sebagai kesalahan kecil yang proporsional, sehingga model terhindar dari lonjakan kurva yang ekstrem. Kontradiksi grafis dan numerik ini memberikan *insight* yang krusial: pendekatan klasifikasi lebih responsif dalam mendeteksi indikasi kantuk namun rentan menerima penalti kerugian besar akibat pergeseran fitur minor di sekitar ambang batas. Sementara itu, pendekatan regresi menawarkan stabilitas pelatihan yang sangat baik, namun harus mengorbankan sensitivitas deteksi bahaya karena sifat tebakannya yang cenderung mencari aman.

4.5.2 Analisis Efektivitas Mekanisme *Selective State Space*

Keberhasilan model dalam mencapai rata-rata akurasi yang kompetitif dengan jumlah parameter yang sangat minimal, yaitu hanya sebesar 62.930 unit, merupakan bukti kuat efektivitas mekanisme *Selective State Space* yang menjadi inti dari arsitektur Mamba. Berbeda dengan model RNN tradisional yang sering mengalami degradasi performa pada urutan data yang panjang, Mamba mampu menjaga aliran informasi temporal penting dari sinyal EEG/EOG sepanjang 15.360 sampel secara efisien.

Kemampuan Mamba dalam melakukan kompresi informasi secara linear menjadi kunci utama mengapa model yang dikembangkan tetap mampu menghasilkan performa yang stabil di tengah tingginya variabilitas antar-subjek (*inter-subject variability*) pada *dataset* DROZY. Hal ini menunjukkan bahwa arsitektur Mamba tidak hanya unggul secara teoritis, tetapi juga sangat aplikatif dalam menangani data sinyal biologis yang kompleks dan *time-series*.

4.5.3 Analisis Variansi dan Pengaruh Karakteristik Subjek

Berdasarkan Tabel 4.3 dan Tabel 4.4, hasil evaluasi menggunakan *5-Fold Cross-Validation* menunjukkan adanya variansi performa yang cukup tinggi antar-*fold*. Pada pendekatan klasifikasi, nilai F1-Score berfluktuasi dari angka tertinggi 0,7512 (Fold 1) hingga angka terendah 0,3667 (Fold 5), dengan standar deviasi keseluruhan mencapai $\pm 0,1319$. Tingginya angka variansi ini membuktikan bahwa kemampuan prediksi model masih sangat dipengaruhi oleh subjek yang sedang dievaluasi.

Investigasi lebih lanjut terhadap penurunan performa yang drastis pada Fold 4 dan Fold 5 mengindikasikan adanya pengaruh kuat dari *inter-subject variability*. Sinyal fisiologis seperti EEG sangat unik untuk setiap individu. Subjek-subjek yang berada pada himpunan uji di Fold 5 kemungkinan besar memiliki pola transisi gelombang otak yang berbeda dari distribusi mayoritas subjek pada data latih.

Dari perspektif implementasi di dunia nyata, tingginya variansi ini memberikan dampak langsung terhadap keandalan sistem. Model yang ada saat ini belum sepenuhnya tangguh (*robust*) untuk digeneralisasikan kepada pengguna baru tanpa adanya penyesuaian. Hal ini mengimplikasikan bahwa sistem deteksi kantuk yang optimal tidak hanya membutuhkan arsitektur yang ringan seperti Mamba, tetapi ke depannya juga memerlukan fase kalibrasi singkat untuk mempelajari sinyal dasar dari masing-masing subjek baru, atau membutuhkan pelatihan dengan dataset berskala jauh lebih besar untuk mencakup seluruh kemungkinan variasi sinyal otak manusia.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil pengujian, evaluasi, dan analisis yang telah dilakukan pada penelitian ini, dapat ditarik kesimpulan untuk menjawab rumusan masalah sebagai berikut:

1. Arsitektur Mamba berhasil dirancang untuk memproses sinyal EEG dan EOG dari dataset DROZY. Untuk menjaga objektivitas studi perbandingan, pemodelan ini diinisiasi secara terpisah antara skenario regresi dan klasifikasi tanpa adanya pembagian bobot (*weight sharing*). Secara teknis, model ini terbukti sangat efisien dengan total 62.930 parameter, beban komputasi 0,01 GFLOPS, dan memori *runtime* 1,81 MB.
2. Hasil pengujian *5-Fold Cross-Validation* membuktikan adanya perbedaan karakteristik performa yang kontras antara kedua pendekatan. Cabang regresi (Akurasi 57,51%; F1-Score 0,4676) memiliki kecenderungan yang sangat berhati-hati sehingga efektif dalam meminimalkan alarm palsu, namun lambat dalam mendeteksi gejala awal kantuk. Sebaliknya, cabang klasifikasi biner (Akurasi 66,00%; F1-Score 0,5844) bersifat lebih agresif dan andal dalam menangkap kondisi mengantuk. Tingginya standar deviasi performa pada kedua model turut membuktikan bahwa akurasi prediksi sangat bergantung pada variabilitas sinyal antar-subjek. Meskipun model klasifikasi rentan terhadap *overfitting* pada batas transisi fase kantuk, pendekatannya dinilai lebih relevan untuk memprioritaskan keselamatan berkendara.

5.2 Saran

Berdasarkan hasil penelitian dan analisis yang telah dilakukan, terdapat beberapa saran yang dapat diajukan untuk pengembangan penelitian selanjutnya guna meningkatkan performa dan fungsionalitas sistem deteksi kantuk:

1. Optimasi Pembobotan Multi-Task Learning: Melakukan eksperimen lebih lanjut pada pembobotan *Loss Function* antara cabang regresi dan klasifikasi untuk menemukan titik keseimbangan terbaik antara akurasi dan stabilitas prediksi.
2. Penggunaan Dataset Skala Besar: Disarankan untuk melakukan pelatihan ulang menggunakan *dataset* dengan jumlah sampel dan durasi rekaman yang lebih besar. Keterbatasan jumlah data pada *dataset* DROZY menjadi kendala utama bagi model untuk mempelajari pola transisi kantuk secara lebih mendalam. Penambahan volume data diharapkan dapat meningkatkan kemampuan generalisasi model Mamba sehingga performa prediksi menjadi lebih stabil dan akurat.
3. Eksplorasi Fitur Domain Frekuensi sebagai Input Pelengkap: Sebagai alternatif augmentasi representasi, penelitian selanjutnya dapat mengeksplorasi penggabungan fitur *Power Spectral Density* (PSD) dari pita frekuensi EEG (delta, theta, alpha, beta) sebagai fitur tambahan yang dikoncatenasikan ke vektor representasi setelah AdaptiveAvgPool1D, tanpa menggantikan input sekuensial mentah. Pendekatan hybrid ini mempertahankan justifikasi penggunaan Mamba sebagai sequential model, sekaligus memberikan informasi frekuensi eksplisit yang relevan untuk klasifikasi kantuk.

DAFTAR PUSTAKA

- [1] Badan Pusat Statistik (BPS). *Statistik Transportasi Darat 2023. Land Transportation Statistics 2023*. Vol. 9. Katalog/Catalogue: 8302004. Nomor Publikasi/Publication Number: 06100.24058. Jakarta: Badan Pusat Statistik/BPS-Statistics Indonesia, 2024. ISBN: 2598-5612.
- [2] John Gaspar and Cher Carney. *Drowsiness and Decision Making During Long Drives: A Driving Simulation Study*. Tech. rep. Washington, D.C.: AAA Foundation for Traffic Safety, 2023.
- [3] Yaman Albadawi, Maen Takruri, and Mohammed Awad. “A Review of Recent Developments in Driver Drowsiness Detection Systems”. *Sensors* 22.2069 (2022).
- [4] Hongyi Wang et al. “Electroencephalogram-Based Approaches for Driver Drowsiness Detection and Management: A Review”. *Sensors* 22.3 (2022), p. 1100.
- [5] Quentin Massoz et al. “The ULg multimodality drowsiness database (called DROZY) and examples of use”. *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE. 2016, pp. 1–9.
- [6] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. *arXiv preprint arXiv:2312.00752* (2023).
- [7] Yia Zhou et al. “EEGMamba: An EEG foundation model with Mamba”. *Neural Networks* (2025). PubMed: 40714477.
- [8] Jingjing Chen et al. “A Large Finer-grained Affective Computing EEG Dataset” (2023).
- [9] Garv Siddhad, Shounak Dey, and Puspita P Roy. “DrowzEE-G-Mamba: Leveraging EEG and State Space Models for Driver Drowsiness Detection”. *arXiv preprint arXiv:2408.16145* (2024).

- [10] Daniel Sommer and Martin Golz. “Evaluation of PERCLOS Based Current Fatigue Monitoring Technologies”. *IEEE Engineering in Medicine and Biology Society* (2015), pp. 4456–4459.
- [11] Jeremy D. Slater. “A definition of drowsiness: One purpose for sleep?” *Medical Hypotheses* 71 (2008), pp. 641–644.
- [12] Xin Zhou et al. “BiT-MamSleep: Bidirectional Temporal Mamba for EEG Sleep Staging”. *arXiv preprint arXiv:2411.01589* (2024).
- [13] Shengli Cao et al. “Optimized driver fatigue detection method using multimodal neural networks”. *Scientific Reports* 15.1 (2025), p. 12240. 2045-2322.
- [14] Rami N. Khushaba et al. “Driver drowsiness classification using fuzzy wavelet-packet-based feature-extraction algorithm”. *IEEE Transactions on Biomedical Engineering* 58.1 (2011), pp. 121–131.
- [15] Anuva Chowdhury et al. “Sensor Applications and Physiological Features in Drivers’ Drowsiness Detection: A Review”. *IEEE Sensors Journal* 18.8 (2018), pp. 3055–3067.
- [16] Igor Stancin, Mario Cifrek, and Alan Jovic. “A Review of EEG Signal Features and Their Application in Driver Drowsiness Detection Systems”. *Sensors* 21.11 (2021). 1424-8220.
- [17] Yingying Jiao et al. “Driver sleepiness detection from EEG and EOG signals using GAN and LSTM networks”. *Neurocomputing* 408 (2020), pp. 100–111. 0925-2312.
- [18] Jaakko Malmivuo and Robert Plonsey. *Bioelectromagnetism: principles and applications of bioelectric and biomagnetic fields*. Oxford university press, 1995.
- [19] Mengchuang Song et al. “A new method for muscular visual fatigue detection using electrooculogram”. *Biomedical Signal Processing and Control* 58 (2020), p. 101865. 1746-8094.

- [20] Thum Chia Chieh et al. "Development of vehicle driver drowsiness detection system using electrooculogram (EOG)". *2005 1st International Conference on Computers, Communications, Signal Processing with Special Track on Biomedical Engineering*. 2005, pp. 165–168.
- [21] Sung-Phil Kim. "Preprocessing of EEG". *Computational EEG Analysis: Methods and Applications*. Ed. by Chang-Hwan Im. Singapore: Springer Singapore, 2018, pp. 15–33. ISBN: 978-981-13-0908-3.
- [22] Aleš Procházka et al. "Multi-channel EEG signal segmentation and feature extraction". *2010 IEEE 14th International Conference on Intelligent Engineering Systems*. 2010, pp. 317–320.
- [23] Henry Candra et al. "Investigation of window size in classification of EEG-emotion signal with wavelet entropy and support vector machine". *2015 37th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. 2015, pp. 7250–7253.
- [24] Andrea Apicella et al. "On the effects of data normalization for domain adaptation on EEG data classification". *Engineering Applications of Artificial Intelligence* 123.Part A (2023), p. 106205.
- [25] Mohammad Akyash, Hoda Mohammadzade, and Hamid Behroozi. "DTW-MERGE: A NOVEL DATA AUGMENTATION TECHNIQUE FOR TIME SERIES CLASSIFICATION". *arXiv preprint arXiv:2103.01119* (2021).
- [26] Woo-Hyeon Kim et al. "Time-Series Data Augmentation for Improving Multi-Class Classification Performance". *International Journal on Advanced Science Engineering Information Technology* 14.3 (2024), pp. 887–893.
- [27] Guillermo Iglesias et al. "Data Augmentation techniques in time series domain: A survey and taxonomy". *arXiv preprint arXiv:2206.13508* (2022).

- [28] Meenu Bhagat and Dr. Brijesh Bakariya. “A Comprehensive Review of Cross-Validation Techniques in Machine Learning”. *International Journal for Sciences and Technology* (2025).
- [29] Ilias Tougui, Abdelilah Jilbab, and Jamal El Mhamdi. “Impact of the Choice of Cross-Validation Techniques on the Results of Machine Learning-Based Diagnostic Applications.” *Healthcare Informatics Research* (2021).
- [30] Nikhil Ketkar and Eder Santana. *Deep learning with Python*. Vol. 1. Springer, 2017.
- [31] Francois Chollet and François Chollet. *Deep learning with Python*. simon and schuster, 2021.
- [32] Syama Sundar Rangapuram et al. “Deep State Space Models for Time Series Forecasting”. *Neural Information Processing Systems* (2018).
- [33] IBM. *Mamba: A Deep Learning Model for Linear-Time Sequence Modeling*. <https://www.ibm.com/think/topics/mamba-model>. Diakses pada: 6 Desember 2025. 2024.
- [34] Trong Huy Phan and Kazuma Yamamoto. “Resolving Class Imbalance in Object Detection with Weighted Cross Entropy Losses”. *arXiv.org* (2020).
- [35] S. Sathyanarayanan and B. Roopashri Tantri. “Confusion Matrix-Based Performance Evaluation Metrics”. *African Journal of Biomedical Research* 27.4s (2024), pp. 4023–4031.
- [36] Damir Krstinić et al. “Multi-label classifier performance evaluation with confusion matrix”. *Computer Science & Information Technology (CS & IT)* (2020), pp. 01–14.
- [37] Muthuramalingam Sivakumar, Sudhaman Parthasarathy, and Thiagarajan Padmapriya. “A simplified approach for efficiency analysis of machine learning algorithms”. *PeerJ Computer Science* 10 (2024), e2418.

- [38] Gaurav Menghani. “Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better”. *ACM Comput. Surv.* 55.12 (Mar. 2023). 0360-0300.
- [39] Ali Borji. “Enhancing sensor resolution improves CNN accuracy given the same number of parameters or FLOPS”. *CoRR* abs/2103.05251 (2021). arXiv: [2103.05251](https://arxiv.org/abs/2103.05251).
- [40] Yuxi Zhao et al. “Research on road surface damage detection based on SEA-YOLO v8”. *PLOS ONE* 20.6 (June 2025), pp. 1–26.
- [41] Cort J Willmott and Kenji Matsuura. “Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance”. *Climate Research* 30 (2005), pp. 79–82.
- [42] Aymen Zayed et al. “A Novel Hybrid Approach for Drowsiness Detection Using EEG Scalograms to Overcome Inter-Subject Variability”. *Sensors* 25.17 (2025), p. 5530.